

HispaLIS

Memoria técnica de un Sistema de Información de Laboratorio simulado sobre HL7 FHIR
R5

Andrés Ojeda Rodríguez

16 de agosto de 2026

Índice general

1. Aviso previo: qué es HispaLIS y qué NO es	3
1.1. Las cinco cosas que hay que saber antes de seguir leyendo	3
1.2. Para quién es este documento	4
1.3. Qué NO es, dicho por su nombre	4
2. Alcance, no-objetivos y los tres hitos	5
2.1. Lo que hay dentro	5
2.2. Lo que queda fuera, y por qué	5
2.3. Los tres hitos	6
2.4. Qué demuestra cada criterio del último hito	6
3. La arquitectura	8
3.1. Dos planos de entrada que no se mezclan	8
3.2. FHIR es un formato de borde, no el modelo de dominio	9
3.3. Read-your-writes es norma, no rendimiento	10
3.4. El reconciliador: la vía de recuperación es oficial, no un script de emergencia	11
4. Los invariantes: lo que un mantenedor no puede romper sin saberlo	12
4.1. Los nueve invariantes del proyecto	12
4.2. Los invariantes de negocio que FHIR no puede expresar	13
5. Los componentes, uno a uno	14
5.1. El stack, y qué hace cada pieza	14
5.2. backend/ — el dominio, la API y la proyección	14
5.3. integracion/ — el motor de canales HL7 v2	16
5.4. web-profesional/ — Angular	16
5.5. app-ciudadano/ — Flutter	17
5.6. simuladores/ — el generador y los terceros	17
5.7. terminologia/ — el servidor y su cargador	18
5.8. infra/ — la orquestación	18
6. La superficie de interoperabilidad	19
6.1. La API FHIR	19
6.2. La guía de implementación publicada	19
6.3. Identificadores y nombres españoles	20
6.4. Las extensiones: solo cuando no existe elemento estándar	21
6.5. R5 no es R4: la tabla que hay que mirar antes de escribir el primer mapeo	22
6.6. La terminología	23
6.7. Los canales HL7 v2	25
6.7.1. MLLP: una trampa documental de tres capas	26

7. El modelo de seguridad	27
7.1. Son dos capas, y no son intercambiables	27
7.2. La autorización SMART	27
7.3. El consentimiento: la mitad que no se puede delegar	28
7.4. La exportación masiva: una puerta distinta	28
7.5. Los clientes y sus credenciales	28
7.6. La auditoría: todo se registra, y sin PHI	29
8. Cómo se levanta y cómo se recorre	31
8.1. Requisitos	31
8.2. Levantar la pila	31
8.3. El circuito, de extremo a extremo	32
8.4. Construir la guía en local	34
8.5. Comandos por componente	34
8.6. Integración continua	35
9. Los números finales	36
9.1. Qué se validó, y con qué	36
9.2. Los tests	36
9.3. Los apellidos y la eñe, que son un caso de prueba y no un detalle	37
9.4. Cuánto tarda cada cosa	37
9.5. El estado de los siete workflows	38
10.Las decisiones	39
10.1. D1 a D23	39
10.2. Los cuarenta y cinco ADR	40
11.Trampas medidas: el conocimiento que no es decisión ni número	43
11.1. De la guía y de la cadena FSH	43
11.2. De los servidores	44
11.3. De la seguridad y del cliente	45
11.4. De la cadena de construcción y del entorno	45
11.5. Lo que solo se ve ejecutando	46
12.Qué es demostración y NO vale para producción	48
12.1. Credenciales y secretos	48
12.2. Disponibilidad y escala	48
12.3. Superficie expuesta	49
12.4. Los terceros son simulados	49
12.5. Y lo que se dice en la propia guía	49
13.Qué queda abierto	50
13.1. Bloqueado por datos que no se pueden redistribuir	50
13.2. Modelado que se dejó fuera a propósito	51
13.3. Infraestructura montada para demostrar	51
13.4. Cliente y pantallas	51
13.5. Cobertura y garantías que se saben incompletas	52
13.6. El siguiente trabajo natural, si alguien continúa	52
14.Procedencia de este documento	53

Capítulo 1

Aviso previo: qué es HispaLIS y qué NO es

HispaLIS no es un producto sanitario. No es un sistema en uso. No ha tratado nunca datos de una persona real y no debe tratarlos.

Es una **simulación** de un **SIL** —Sistema de Información de Laboratorio, que es como se llama en España a lo que la literatura anglosajona llama *LIS*— para un laboratorio clínico privado de tamaño medio en Sevilla, construida sobre **HL7 FHIR R5 (5.0.0)**. El propósito es didáctico y demostrativo: atravesar los ejes reales de la interoperabilidad sanitaria —una guía de implementación propia con terminología, una API FHIR conforme, un puente HL7 v2 sobre MLLP, un bus de eventos, SMART on FHIR y una obligación legal española implementada— sin degenerar en una historia clínica electrónica en miniatura.

Hispalis era el nombre romano de Sevilla; **LIS** es el acrónimo internacional. La mayúscula hace visible el juego.

1.1. Las cinco cosas que hay que saber antes de seguir leyendo

1. **Todos los datos son sintéticos.** Los produce **simuladores/generador/**. No hay, no ha habido y no puede haber datos reales de pacientes en ningún entorno de este proyecto. Es un invariante del proyecto (§4.1, invariante 5), no una recomendación.
2. **La infraestructura es de demostración y no vale para producción.** Contraseñas de juguete, certificados autofirmados, instancia única en todo lo asíncrono, un APK firmado con la clave de depuración. La lista completa y sin suavizar está en el capítulo 12, y merece leerse antes de enseñarle esto a nadie.
3. **La normativa española que implementa se modela de forma verosímil, no fiel.** La declaración de enfermedades de declaración obligatoria (EDO) a Salud Pública existe y es un requisito legal real también para los centros privados, pero el contrato de la aplicación **Redalerta** no es público: lo que se manda es el **Task** que publica esta misma guía. Y hay una diferencia que conviene decir en voz alta: **una declaración EDO real lleva filiación** —Salud Pública tiene que poder localizar al caso para la encuesta epidemiológica— y **esta no lleva ninguna**, porque el destinatario es simulado y de aquí no salen datos de persona hacia ningún sistema externo.
4. **Las URIs canónicas de la guía de implementación son propias, no oficiales.** España no tiene un juego oficial consolidado para estos espacios de nombres. Se definieron propias bajo <https://aojeda006.github.io/HispaLIS/fhir>, se publican y **se documenta que son propias**. Dos de ellas —DNI/NIE y CIP-SNS— se adoptaron del Ministerio de Sanidad; ver D21 en §10.1.

5. **ISO 15189 está fuera de alcance como requisito.** Es una acreditación voluntaria, no una obligación legal; la obligación real de un laboratorio andaluz es la autorización sanitaria del Decreto 69/2008. Aquí se cita solo como justificación de las decisiones de trazabilidad (Provenance de quién validó, AuditEvent de quién accedió).

1.2. Para quién es este documento

Para alguien técnico —desarrollador o arquitecto— que no ha participado en la construcción, que puede no saber FHIR, y que tiene que entender **qué es esto, por qué está hecho así y qué no es**.

Es **autosuficiente**: no remite a ningún documento de trabajo que ya no exista. Lo que sigue vivo en el repositorio y amplía lo de aquí es `docs/disenio.md` (el porqué de las decisiones, con las fuentes normativas españolas citadas), `docs/adr/` (cuarenta y cinco decisiones de arquitectura, una por fichero) y `README.md` (la puerta de entrada operativa).

1.3. Qué NO es, dicho por su nombre

- **No es una historia clínica electrónica.** Modela un solo proceso, cerrado: petición, extracción, espécimen, analizador, resultado, validación facultativa, informe, entrega. Todo lo que no cabe en ese proceso queda fuera a propósito.
 - **No está conectado a ningún sistema real.** Ni a Diraya ni a su Módulo de Pruebas Analíticas, ni a la Historia Clínica Digital del SNS, ni a Receta XXI, ni al CMBD. El HIS, el analizador, el receptor de notificaciones y el servicio de Salud Pública son **simulados** y viven en este mismo repositorio.
 - **No es un servidor de terminología.** Pregunta a uno, con las cuatro operaciones estándar de FHIR, y ese servidor es intercambiable.
 - **No implementa el Esquema Nacional de Seguridad.** Obliga al sector público y a sus proveedores; un laboratorio privado puro sin concierto no está sujeto.
 - **No es un motor de integración comercial.** No es Mirth. Los canales son código Java que se despliega por el mismo circuito que el resto y se revisa como código.
-

Capítulo 2

Alcance, no-objetivos y los tres hitos

2.1. Lo que hay dentro

Eje	Qué se construyó
Guía de implementación	12 perfiles FHIR R5, 1 extensión propia, 6 <code>CodeSystem</code> , 10 <code>ValueSet</code> , 1 <code>ConceptMap</code> , 1 <code>SubscriptionTopic</code> , 1 <code>SearchParameter</code> , 1 <code>OperationDefinition</code> , 38 ejemplos — publicada y navegable
Núcleo y API	Dominio propio con sus agregados e invariantes; API FHIR R5 servida por HAPI FHIR JPA como proyección , escrita en la misma transacción
Interoperabilidad v2	Motor propio con HAPI HL7v2: ADT^A01/A08, OML^021, ORU^R01 entrantes por MLLP sobre TLS; ORU^R01 saliente
Terminología	Servidor HAPI con subconjuntos curados de LOINC y THO, catálogo local propio y <code>ConceptMap</code> hacia LOINC; cuatro operaciones estándar y ninguna propietaria
Eventos	<code>outbox</code> transaccional y relay hacia Kafka con Schema Registry, clave de partición por paciente
Identidad	Keycloak con SMART on FHIR: EHR launch, standalone launch con PKCE y Backend Services con <code>private_key_jwt</code>
Clientes	Web profesional en Angular y app del ciudadano en Flutter
Lo que solo existe en R5	<code>SubscriptionTopic</code> + <code>Subscription</code> entregando <code>id-only</code> , <code>Observation.triggeredBy</code> para reflejas, <code>Coverage.kind</code>
Obligación legal	Detección de EDO por código y declaración a Salud Pública con acuse, plazo y estados
Datos masivos	<code>\$export</code> de Bulk Data sobre un <code>Group</code> , con cohorte seudonimizada que caduca y se borra
Trazabilidad	<code>AuditEvent</code> de toda lectura y escritura, sin una palabra de más

2.2. Lo que queda fuera, y por qué

Fuera	Motivo
ISO 15189 como requisito	Acreditación voluntaria; la obligación real es la autorización sanitaria (D17)
Conexión al MPA de Diraya	Su contrato de interfaz no es público: simularlo daría falso realismo y no se podría validar (D8)
HCDSNS / Nodo SNS	Es el nodo entre comunidades del sector público; un privado no se conecta
Receta XXI	Prescripción farmacéutica, no laboratorio
CMBD / RAE-CMBD	Registro de hospitalización, no de laboratorio ambulatorio
ENS (RD 311/2022)	Obliga al sector público y a sus proveedores. Sí aplicaría con concierto
Un <code>Bundle transaction</code> que escriba por la API	La puerta está cerrada a propósito: ese camino de HAPI llama a las DAO sin pasar por el núcleo. Ver D22 en §10.1

2.3. Los tres hitos

El troceado fue **vertical**: cada hito atraviesa de cliente a base de datos y queda demostrable por sí mismo. No hubo un hito «de capa de persistencia» ni uno «de interfaz».

Hito	Cerrado	Qué añadió
1 — el circuito básico	2026-08-06	Petición, espécimen, resultado e informe de extremo a extremo. Sin Kafka, sin v2 y sin Keycloak. La guía FHIR propia publicada, la web del profesional y el generador de datos sintéticos
2 — la interoperabilidad de verdad	2026-08-08	Los tres huecos de dominio (anulación de línea, validación con Provenance , outbox), el puente HL7 V2.5.1 sobre MLLP/TLS con almacén de mensajes, DLQ y reproceso idempotente, Kafka con Schema Registry, el reconciliador, el servidor de terminología, SMART on FHIR y la app del ciudadano
3 — lo que solo existe en R5, lo masivo y lo legal	2026-08-12	SubscriptionTopic/Subscription , reflejas con triggeredBy , umbrales críticos y doble validación, notificador EDO, Bulk Data \$export con Group y AuditEvent completo

Cincuenta y uno de los cincuenta y dos ítems del plan quedaron cerrados. El que falta es el **42** —cargar los códigos SNOMED CT del catálogo—, bloqueado por una licencia que no permite redistribución; el capítulo 13 lo explica entero.

2.4. Qué demuestra cada criterio del último hito

Cada fila es un criterio de aceptación con la prueba concreta de que se cumple. «En vivo» significa contra la pila levantada con `docker compose`, con la seguridad encendida, no contra un doble de pruebas.

#	Criterio	Prueba
42	Los códigos SNOMED del SNS cargados	No cumplido. La Edición Española no se redistribuye. El hueco está modelado y el cargador avisa; sin el fichero no se carga (§13.1)

#	Criterio	Prueba
43	Umbrales críticos y reflejas en el catálogo, no en el código	CatalogoPruebas.fsh declara #TSH ~property[prueba-refleja] = #T4L. En vivo: la refleja aparece sin una sola condición escrita en Java
44	SubscriptionTopic + Subscription entregando	En vivo: 201 con testigo system/Subscription.crs, un receptor real acusando y \$status devolviendo entregados=4
45	Observation.triggeredBy en la refleja	En vivo: triggeredBy=reflex con el motivo redactado que sale del catálogo
46	Crítico implica doble validación de otro facultativo	En vivo: 200, luego 422 a la misma persona firmando dos veces, luego 200 FINAL con dos Provenance
47	Un resultado EDO validado obliga a notificar	En vivo: el Task se abre solo al validar el positivo. La regla vive en el catálogo, no en un if
48	La declaración sale y se acusa	En vivo: ACUSADA con número de registro del SVEA; el libro de Salud Pública con tres declaraciones y cero fuera de plazo
49	\$export autorizado y caducable	En vivo: 202, manifiesto, tres NDJSON, DELETE, 404. Los dos ámbitos concedidos y retirados al terminar
50	AuditEvent completo y sin PHI	En vivo: cincuenta trazas, cero PHI, cero entity.query. Una búsqueda por número de historia no deja el criterio en ningún sitio

Los doce criterios del hito 1 y los veinticinco ítems del hito 2 se cerraron con el mismo método: cada uno con una prueba ejecutable, y los invariantes de negocio con un test en rojo antes de escribir el código que lo pone en verde.

Capítulo 3

La arquitectura

3.1. Dos planos de entrada que no se mezclan

	Plano de aplicaciones	Plano de sistemas
Quién entra	Web profesional, app del ciudadano, terceros	HIS de la clínica, analizadores
Formato	FHIR R5, solo	HL7 V2.5.1, solo
Transporte	HTTPS	MLLP sobre TLS
Autorización	SMART on FHIR (<i>patient/</i> , <i>user/</i> , <i>system/</i>)	Credenciales del canal y red confinada
Entra por	La API FHIR	El motor de integración

HL7 v2 no entra por el frontal —un navegador no habla MLLP, que es un protocolo de socket con *framing* propio— y **no llega a la API FHIR sin traducir**. El motor de integración *es* el punto de conversión, explícito y auditable. Si los dos contratos se funden en una sola puerta, el mapeo deja de poder auditarse, que es exactamente el fallo que el motor existe para evitar.

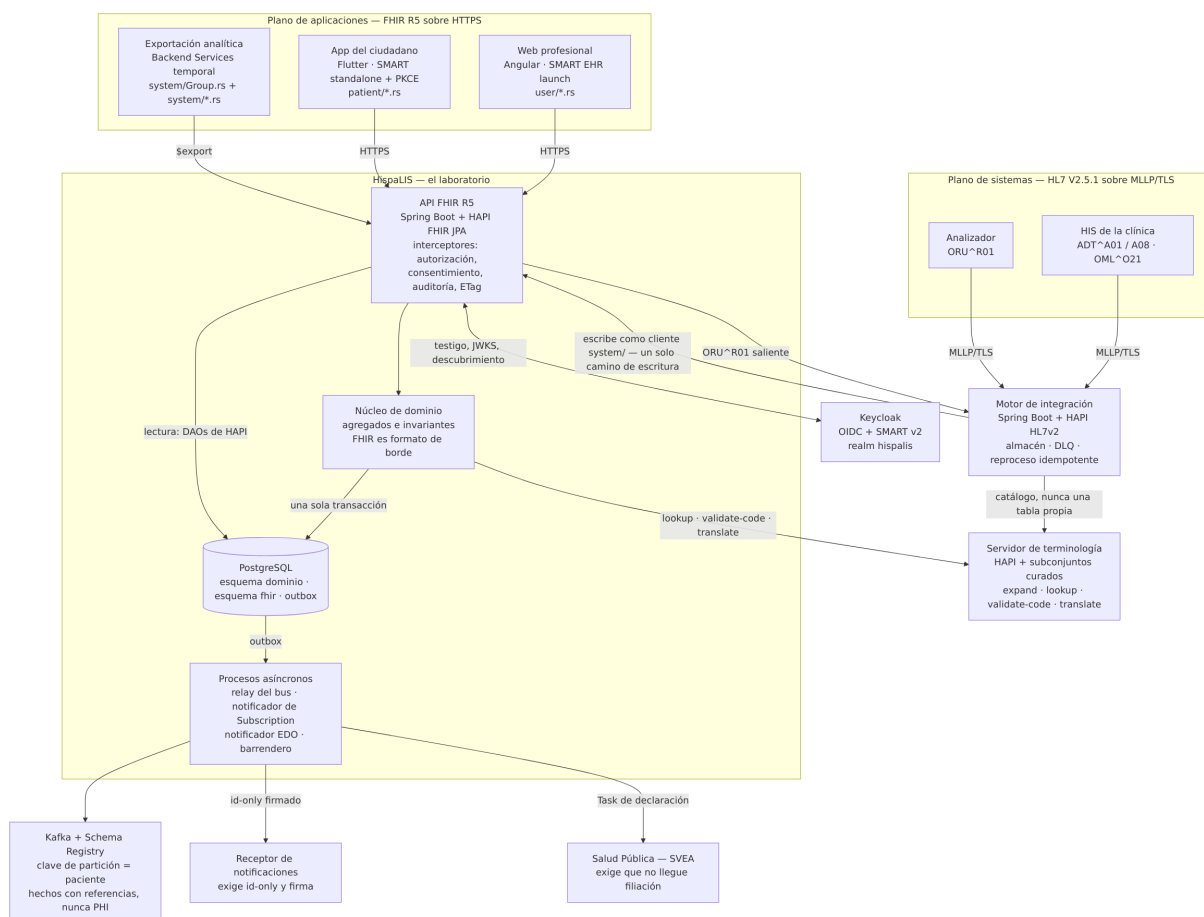


Figura 3.1: Mapa de componentes y cómo se hablan entre ellos

3.2. FHIR es un formato de borde, no el modelo de dominio

Ésta es la decisión estructural del proyecto (D3), y todo lo demás cuelga de ella.

El error clásico en un proyecto FHIR es persistir los recursos FHIR como si fueran entidades del negocio. No lo son: son un **contrato de intercambio**, con opcionalidad enorme, $0..*$ por todas partes y **ninguno** de los invariantes del laboratorio. Un **Observation** de FHIR no sabe que un espécimen rechazado no puede producir un resultado, ni que un valor crítico exige dos firmas de personas distintas.

Así que el sistema tiene **dos modelos, y no coinciden**:

- **El dominio** —esquema dominio de PostgreSQL— es la **fuentes de verdad**. Tiene agregados propios (Petición, Especimen, Resultado, Informe, Notificación EDO) con sus invariantes, y no sabe nada de FHIR.
- **La proyección** —esquema *fhir*, gestionado por HAPI FHIR JPA— es lo que se publica. De ahí salen las lecturas, la búsqueda, la paginación, `_history`, `$validate` y `$export`.

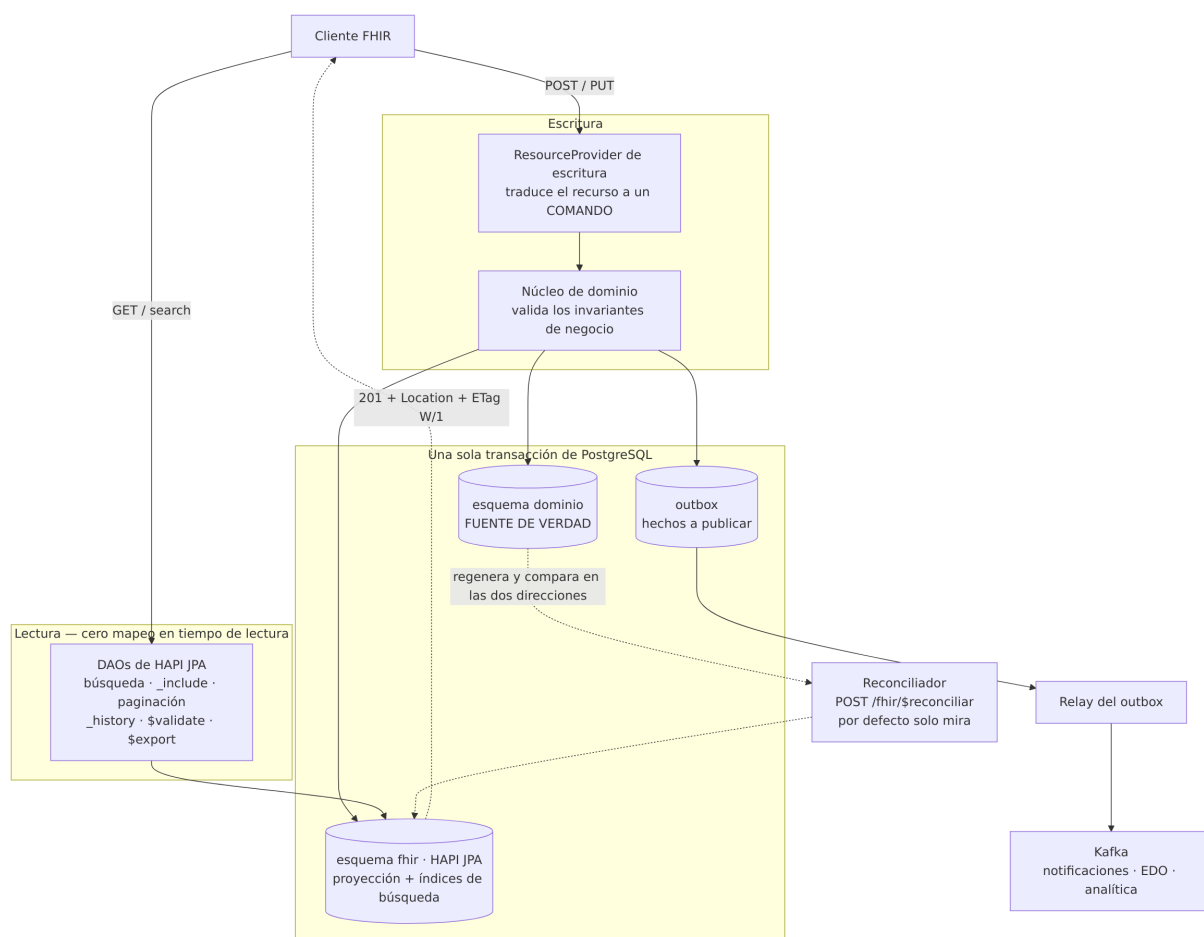


Figura 3.2: El fork estructural: escritura por comando, lectura por proyección, hechos fuera del camino de lectura

En **escritura**, un POST o un PUT llega al **ResourceProvider**, que traduce el recurso a un **comando** y se lo entrega al núcleo. El núcleo valida los invariantes de negocio y, en **una sola transacción de PostgreSQL**, escribe tres cosas: el dominio, la proyección FHIR (llamando a las DAO de HAPI, para que se pueblen los índices de búsqueda) y el **outbox** de hechos a publicar.

En **lectura** no hay mapeo: la petición va directa a las DAO de HAPI y de ahí al esquema **fhir**. Coste de traducción en tiempo de lectura: cero.

Los **hechos** salen del **outbox** por un relay, fuera del camino de lectura. **Kafka no alimenta el modelo de lectura**. Si alguna vez una lectura de la API llega a depender de que un consumidor haya procesado algo, es que se ha roto el punto siguiente.

3.3. Read-your-writes es norma, no rendimiento

FHIR REST exige poder leer lo que acabas de escribir. El servidor devuelve 201 Created con Location: Observation/123 y un ETag (W/"1"); si la proyección fuese asíncrona, el GET inmediato a ese Location devolvería 404. **Eso no es una latencia molesta: es incumplir la norma.**

Por eso la proyección se escribe **síncrona y en la misma transacción**. Hay un test automatizado que hace exactamente eso —escribe y lee acto seguido— y falla si alguien decide algún día que la proyección puede ir «un poquito por detrás».

El riesgo de esta decisión está asumido y registrado (adr-0002, adr-0012): escribir en el esquema

de HAPI dentro de la transacción del dominio ata el proyecto a sus DAO.

Y hay un corolario que parecía gratis y no lo era. Que las dos escrituras ocurran en la misma transacción garantiza que estén las dos; **no** garantiza que digan lo mismo. La proyección se escribe desde el agregado en memoria y el reconciliador la regenera desde el agregado releído, así que cualquier conversión con pérdida entre los dos —una precisión que el almacén no devuelve igual que la recibió— produce una divergencia permanente que la atomicidad no evita. Pasó, con las marcas de tiempo, y está contado en `adr-0045`.

3.4. El reconciliador: la vía de recuperación es oficial, no un script de emergencia

Con dos escrituras del mismo hecho, dominio y proyección pueden divergir por un fallo de mapeo. La respuesta es `POST /fhir/$reconciliar`, en `aplicacion/reconciliacion/`: recorre el dominio, regenera la proyección y detecta **las dos direcciones** —lo que falta y lo que sobra sin agregado detrás—. Por defecto **solo mira**; `aplicar` es un parámetro explícito.

Exige el ámbito `system/*.cruds` porque **borra** recursos publicados de cualquier tipo, y ese ámbito está definido en el *realm* de Keycloak pero **no asignado a ningún cliente**: dárselo a alguien es un acto deliberado, no la consecuencia de una plantilla.

Medido sobre un laboratorio con volumen (300 pacientes escritos por la API recorriendo el circuito entero): el barrido de revisión tarda 50 segundos y **no escala lineal** —166 ms por paciente con 300, 64 ms con 60—, así que sobre un laboratorio con historia la vía practicable es la acotada, que es para lo que existe el parámetro.

Capítulo 4

Los invariantes: lo que un mantenedor no puede romper sin saberlo

4.1. Los nueve invariantes del proyecto

Son las reglas que gobernaron cada decisión de implementación. Están enunciadas tal como se escribieron; romper cualquiera de ellas es un cambio de arquitectura, no un ajuste.

1. **FHIR es formato de borde, no el modelo de dominio.** Nunca se persisten recursos FHIR como entidades del dominio: el núcleo tiene sus propios agregados e invariantes.
2. **Read-your-writes.** La proyección FHIR se escribe **síncrona, en la misma transacción** que el dominio. Un **GET** inmediato al **Location** de un **201** **debe** devolver el recurso.
3. **Un solo camino de escritura.** Todo lo que entra pasa por la API FHIR, con las mismas validaciones, invariantes y auditoría — incluido el motor de integración, que escribe como cliente `system/`.
4. **La terminología es una caja obligatoria, no un enum.** Nada de `Map<String,String>` de códigos: `CodeSystem`, `ValueSet` y `ConceptMap` desde el día uno. El generador de datos sintéticos consume el **mismo** `CodeSystem` y `ConceptMap` que el sistema, nunca una lista paralela.
5. **Nunca datos reales de pacientes**, en ningún entorno. Solo sintéticos.
6. **Nunca PHI en URLs, logs, trazas, analítica ni en el bus de eventos.** El bus publica **hechos con referencias** (`{ pacienteId, petitionId, observationRef }`), no volcados clínicos.
7. **TDD obligatorio** (rojo, verde, refactor) para el comportamiento de negocio.
8. **Errores en `OperationOutcome`** con el código HTTP correcto. Nunca un **200** con un error dentro.
9. **Todo en español:** documentación, doc-comments, narrativa (`text.div`), `display` y mensajes de usuario. Los identificadores y los términos técnicos estándar, en inglés cuando ésa sea la convención del ecosistema (`ServiceRequest`, `accessionIdentifier`, `Bundle.link`).

El invariante 6 tiene una consecuencia que conviene entender: **un tópico replicado es lo más difícil de borrar que hay** el día que alguien ejerza el derecho de supresión del RGPD. Y hay una tensión legal real detrás: la Ley 41/2002 obliga a conservar la historia clínica un mínimo de cinco años desde el alta de cada proceso, también en la sanidad privada, así que el derecho de supresión **no** puede borrar lo que la ley obliga a conservar. La salida es no haber publicado nunca el dato clínico fuera del sistema que lo custodia.

4.2. Los invariantes de negocio que FHIR no puede expresar

Éstos viven en el núcleo, no en el `ResourceProvider`, y todos se probaron por TDD, en rojo primero:

Agregado	Invariante
Petición	No se cierra con líneas pendientes
Espécimen	Rechazado (unsatisfactory) implica que no puede producir resultado
Resultado	Crítico implica doble validación y notificación obligatoria
Informe	Solo se emite con todas las líneas resueltas
Notificación EDO	Resultado EDO validado implica notificación obligatoria

Tres precisiones que costaron un fallo cada una:

- **El agregado recibe el puerto y pregunta él.** `Resultado.validar(ValoresCriticos, ...)` y `Resultado.obligaADeclarar(CatalogoEdo)`. Pasar un `boolean` ya calculado dejaría la regla en el caso de uso, y con ella fuera del alcance de la siguiente puerta de entrada que aparezca.
- **Firmado no es validado.** Un resultado crítico con una firma está firmado y **no** validado. `estaValidado()` significa «tiene todas las firmas que hacían falta», y cuántas hacían falta se pregunta **una vez** y se graba: ni una caída del servidor de terminología ni un cambio del catálogo pueden alterar una validación ya empezada.
- **Un resultado cualitativo se guarda codificado.** El `text` no gana al código: de ese código depende que se declare una enfermedad, y comparar cadenas para eso es una apuesta (`adr-0034`).

Capítulo 5

Los componentes, uno a uno

5.1. El stack, y qué hace cada pieza

Capa	Elección	Por qué
Dominio y API FHIR	Java 21 + Spring Boot + HAPI FHIR R5	Implementación de referencia de FHIR en Java
Motor de integración	Spring Boot + HAPI HL7v2 (ca.uhn.hl7v2)	Mismo <i>toolchain</i> que el backend; canales como código
Web profesional	Angular 22	Cliente de escritorio del laboratorio
App del ciudadano	Flutter	iOS y Android desde un solo código
Identidad	Keycloak	OIDC estándar, con SMART encima mediante <i>mappers</i>
Eventos	Kafka + Schema Registry	Esquemas Avro versionados, compatibilidad hacia atrás
Datos	PostgreSQL (dos esquemas)	Un solo <i>datasource</i> es lo que hace posible la transacción única
Terminología	HAPI FHIR con subconjuntos curados	Intercambiable: se habla con él por las cuatro operaciones estándar
Datos sintéticos y terceros	Python + Faker es_ES	Generador, HIS, analizador, receptor y SVEA
Perfilado	FSH + SUSHI + IG Publisher	La guía se construye y se publica en cada cambio
Orquestación	Docker Compose	Ocho servicios y seis contenedores de un solo uso

5.2. backend/ — el dominio, la API y la proyección

El componente central. Cuatro capas, y la dirección de las dependencias apunta siempre hacia dentro:

Paquete	Qué contiene
dominio/	Los agregados y sus invariantes: <i>paciente</i> , <i>peticion</i> , <i>especimen</i> , <i>resultado</i> , <i>informe</i> , <i>edo</i> , <i>exportacion</i> , <i>hecho</i> . Sin una sola dependencia de FHIR ni de Spring
aplicacion/	Los casos de uso, uno por operación de negocio, y el reconciliador

Paquete	Qué contiene
fhir/	El borde: los ResourceProvider de cada tipo, los interceptores de seguridad, consentimiento y auditoría, y los puertos hacia la terminología
infraestructura/	Persistencia, bus, notificaciones, exportación, seguridad y el cliente de terminología

Además de la API, el backend aloja tres procesos asíncronos que consumen el **outbox**:

- **El relay del bus** (`infraestructura/bus/`), que publica en Kafka con **clave de partición igual al paciente** y esquema Avro versionado. Entrega **al menos una vez** —exactamente una vez exigiría una transacción distribuida—, así que todo consumidor deduplica por `hechoId`.
- **El notificador de Subscription** (§6.6), que entrega `id-only` firmado con HMAC-SHA256.
- **El notificador EDO** (`infraestructura/edo/`), que consume el **outbox** con su **propio** desplazamiento (`edo.hecho_consumido`) y nunca toca el del relay.

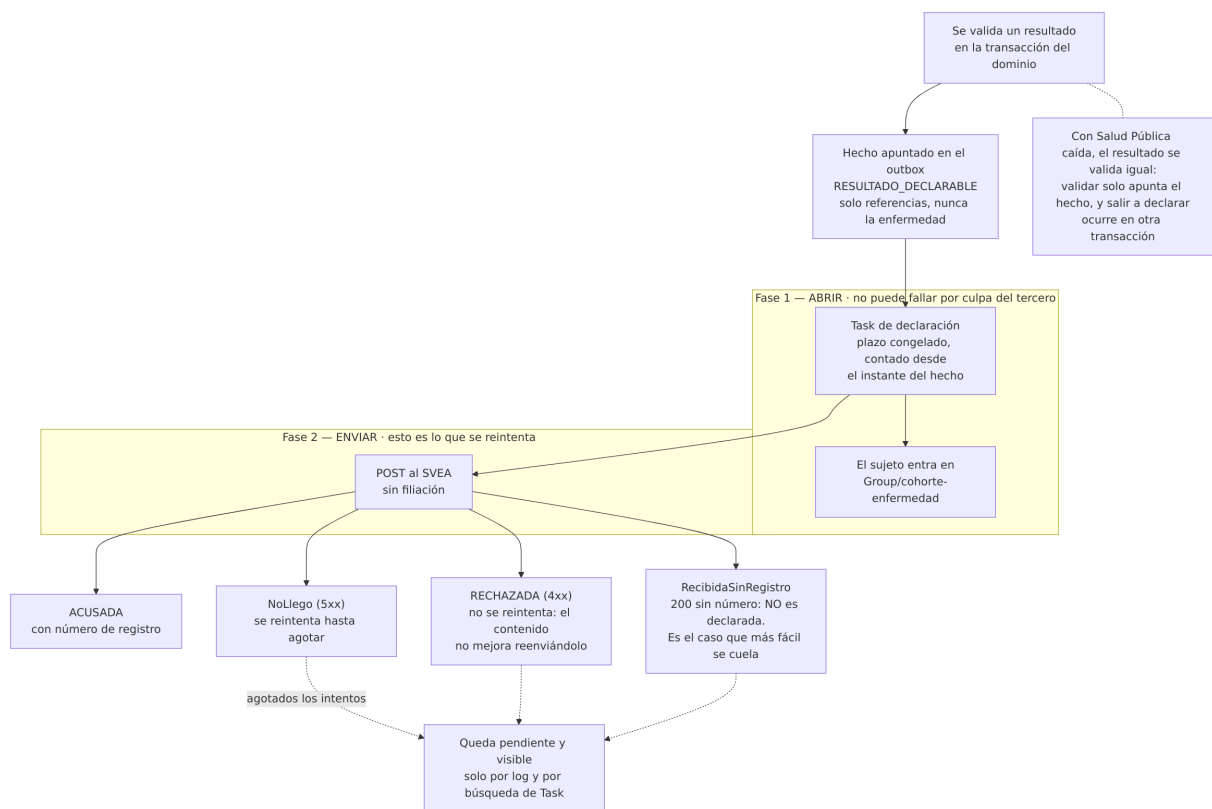


Figura 5.1: La declaración a Salud Pública: dos fases, y la primera no puede fallar por culpa del tercero

La declaración EDO merece detalle porque es la pieza donde una decisión de diseño se ve mejor:

- **Se dispara desde el hecho, no desde un if.** Por eso, con Salud Pública caída, **el resultado se valida igual**: validar solo apunta el hecho, y salir a declarar ocurre en otra transacción.
- **Dos fases, y la primera no puede fallar por culpa del tercero.** *Abrir* (hecho a Task con su plazo) ocurre aunque el destinatario no exista; *enviar* es lo que se reintentará. Fundirlas dejaría la obligación sin registrar justo cuando más falta hace tenerla.

- **Sin acuse no hay declaración**, y está cerrado en tres sitios: **ACUSADA** solo se alcanza por **acusar(Acuse)**, **Acuse** rechaza un número de registro en blanco, y la migración lleva un **CHECK**. Un 200 sin número **no** es una declaración: es el caso que más fácil se cuela, porque a nivel de transporte todo ha ido bien.
- **Cuatro respuestas, tipo sellado, switch sin default**: **Acusada**, **RecibidaSinRegistro**, **Rechazada**, **NoLlego**. Una quinta rompe la compilación en vez de caer en una rama genérica. Un 4xx no se reintenta —el contenido no mejora reenviándolo— y un 5xx sí.
- **El plazo es de la enfermedad, no de la prueba**, y se congela al abrir contando **desde el instante del hecho**: si el notificador estuvo parado seis horas, el plazo legal lleva seis horas corriendo. Una modalidad desconocida o una enfermedad sin plazo declarado no se suponen: se avisa y no se abre nada.

5.3. integracion/ — el motor de canales HL7 v2

Servicio Spring Boot propio con la librería **HAPI HL7v2**, que aporta el listener MLLP, el parser y la generación de acuses. **No es Mirth**: los canales son código, se despliegan por el mismo circuito que el resto y se revisan como código. Nunca se edita un canal en una consola.

Estructura de cada canal: **origen, filtro, transformador, destino**.

Paquete	Qué contiene
mllp/ almacen/	El listener sobre TLS y el <i>framing</i> El mensaje original guardado íntegro, con metadatos indexables (paciente, episodio, MSH-10)
canal/adt, canal/oml, canal/oru saliente/	Los tres canales entrantes El ORU^R01 hacia el HIS cuando el informe se valida
reproceso/ terminologia/	La bandeja de errores y el reproceso idempotente CatalogoDelLaboratorio : ninguna tabla de códigos dentro del motor
infraestructura/seguridad/	AutenticacionDelMotor : SMART Backend Services (§7.5)

Cuatro reglas del canal que no se negocian:

1. **Guardar el mensaje original íntegro antes de tocarlo**. Es lo que hace posible el reproceso, y es lo que se pierde al no usar Mirth.
2. **Deduplicar por MSH-10 en el motor, antes de escribir**.
3. **DLQ y punto de reproceso idempotente**. Reaplicar el mismo mensaje dos veces no puede producir dos volantes; eso está probado, y su test es un test de la decisión D22, no de una utilidad.
4. **Charset declarado en MSH-18 y normalización en la entrada**. Las tildes y la ñe rompen tuberías v2 constantemente (adr-0021).

5.4. web-profesional/ — Angular

La web del profesional del laboratorio: alta de petición y consulta de informe, contra la API FHIR real. Se lanza con **SMART EHR launch**.

- Habla FHIR **R5** y solo **R5**. Cuidado con `ServiceRequest.code`, que en R5 es un `CodeableReference` y no un `CodeableConcept` (§6.5).
- **La paginación se sigue por `Bundle.link[relation=next]`**, nunca construyendo la URL a mano.

- Los errores llegan en `OperationOutcome` y se presentan en español, sin volcar el recurso crudo ni la traza de la excepción.
- **El catálogo de pruebas no se escribe aquí y tampoco se congela al construir:** se le pide al servidor de terminología con `$expand` del `ValueSet` de la guía. Una lista de códigos en TypeScript sería una cuarta versión de la verdad; una copia empaquetada al construir sería la misma verdad con la fecha del último despliegue.
- **Accesibilidad y claridad no son opcionales:** es una herramienta de trabajo clínico, donde confundir un paciente o una unidad tiene consecuencias. La unidad va **siempre** junto al valor, y el rango de referencia visible junto al resultado.

5.5. app-ciudadano/ — Flutter

La app del paciente para consultar sus resultados. Flutter (D13) porque el objetivo declarado eran clientes multiplataforma y en España iOS es aproximadamente la mitad del mercado: una app de resultados solo para Android no cumple la premisa.

- **SMART standalone launch + PKCE**, cliente público, *scopes patient/*.rs*.
- Los testigos van al **almacén seguro de la plataforma**, nunca a `SharedPreferences` ni a un fichero en claro.
- Datos clínicos en caché local: **el mínimo imprescindible y cifrado**, y borrado al cerrar sesión.
- **Un resultado sin contexto asusta:** siempre unidad y rango de referencia junto al valor, y se dice explícitamente cuándo un informe aún no lo ha validado un facultativo.

5.6. simuladores/ — el generador y los terceros

Carpeta	Qué es	Hito
generador/	El generador de datos sintéticos	1
his/	El HIS de la clínica: emite ADT^A01/A08 y OML^O21 por MLLP	2
analizador/	El analizador: emite ORU^R01 por MLLP	2
receptor/	El receptor de notificaciones de Subscription	3
svea/	El servicio de declaraciones de Salud Pública	3

El **generador** es lo que hace que el resto valga algo:

- **Resuelve la terminología contra el mismo servidor que el backend y el motor.** Nunca una lista paralela ni un fichero propio: si se desviara, generaría datos que solo valen para sí mismo y el `ConceptMap` dejaría de estar probado. **Sin servidor no genera**, y hace bien: un corpus con un catálogo a medias es peor que ninguno, porque nadie se entera hasta mirarlo.
- Lo difícil no es la demografía: son los **resultados clínicamente verosímiles** —paneles correlacionados, valores dentro y fuera de rango, y disparos de reflejas que ejerciten `Observation.triggeredBy`—.
- **Localización española real:** apellidos dobles completos, DNI/NIE con dígito de control válido, NUHSA con formato AN más diez dígitos, códigos INE de provincia y municipio.

- **Semilla parametrizable y salida reproducible:** sin reproducibilidad no sirve como arnés de carga ni de pruebas.

El receptor de notificaciones y el SVEA existen porque *un contrato tiene dos lados y solo se demuestra desde los dos*:

- El receptor no arranca sin clave compartida —aceptar una notificación sin firma es aceptarla de cualquiera—, **exige id-only desde el lado que recibe y detecta los huecos de eventNumber**, que es para lo que ese número existe: con entrega «al menos una vez» y un canal que puede caerse, no hay otra forma de enterarse de lo que **no** llegó.
- El SVEA **exige que la declaración no lleve filiación**, y ése es el motivo de que esté aquí y no como un doble dentro del backend. Rechaza con 400 un **contained**, las claves que en FHIR solo cuelgan de una persona (**name, birthDate, address, telecom, photo, gender**) y un **display** sobre una referencia a **Patient, Practitioner** o **RelatedPerson**, que es el nombre con otro nombre. Deduplica por el id del **Task** —el laboratorio reintenta hasta que hay acuse, y un destinatario que no deduplica convierte cada reintento en un caso nuevo en la estadística—, exige plazo (422 si falta) y **registra lo que llega tarde**, porque el plazo no extingue la obligación: la hace tardía. Tiene cuatro modos provocables (**acusa, rechaza, sin-registro, silencio**), que son los cuatro finales que el notificador distingue.

Los dos simuladores v2 hablan V2.5.1 con el juego de caracteres declarado en MSH-18, y **pueden repetir el MSH-10 a propósito**: es la clave de deduplicación del motor, y el camino de duplicados no se prueba de verdad si no hay quien mande el mismo identificador dos veces.

5.7. terminologia/ — el servidor y su cargador

Pieza	Qué es
hapi/application.yaml	La configuración del servidor. La imagen no se construye : es hapiproject/hapi tal cual
cargador/	Lo que sube los subconjuntos curados, por PUT de la API estándar

Lo que no hay, y no puede haber: ni una *release* de terminología licenciada. LOINC, THO y SNOMED viven **fuera del repositorio** y se montan al arrancar el servicio.

5.8. infra/ — la orquestación

El **docker-compose.yml** levanta **ocho servicios** —PostgreSQL, Kafka, registro de esquemas, servidor de terminología, Keycloak, backend, motor y web— más **seis contenedores de un solo uso** que hacen su trabajo y terminan: los tópicos de Kafka, la carga de terminología, las contraseñas de los usuarios de demostración, el almacén de claves del MLLP y los dos que dan permiso de escritura a los volúmenes recién creados (montar un volumen no es poder escribir en él, **adr-0035**).

Dos servicios más van **detrás de un perfil**, y no por comodidad: el **receptor de notificaciones** y el **SVEA** son terceros, y tenerlos siempre arriba daría a entender que el laboratorio depende de que estén. No depende: con el SVEA parado, un resultado declarable se valida igual y su declaración se queda pendiente.

Capítulo 6

La superficie de interoperabilidad

6.1. La API FHIR

- GET /fhir/metadata devuelve un `CapabilityStatement` que declara `fhirVersion 5.0.0` y los perfiles soportados.
- **Concurrencia optimista obligatoria:** un PUT con If-Match de una versión obsoleta contesta 412.
- **Búsqueda y paginación** por `SearchParameter` y `Bundle.link[relation=next]`. Nunca se construye la URL de la página siguiente a mano.
- **Errores en `OperationOutcome`** con el código HTTP correcto; nunca un 200 con un error dentro. Una distinción que se usa a conciencia: una prueba fuera del catálogo es **422, no 400** —el recurso está bien formado, lo que pasa es que el laboratorio no oferta ese análisis, y eso es una regla de negocio—.
- **Interceptores** de autorización, consentimiento, auditoría y ETag/If-Match.
- **El gateway no habla FHIR.** Valida el testigo y enruta. En cuanto se le mete lógica FHIR hay dos servidores FHIR y ninguno conforme.

6.2. La guía de implementación publicada

Se escribe en **FSH**, la compila **SUSHI** y la publica el **IG Publisher** en <https://aojeda006.github.io/HispaLIS/>. Está fijada a `hl7.fhir.r5.core@5.0.0` y declara como dependencia `hl7.fhir.uv.extensions@5.3.0` — en R5 las extensiones salieron del núcleo y viven en un paquete aparte.

La fuente de verdad son los .fsh. Lo que producen SUSHI y el publisher es artefacto generado: no se edita jamás y no se versiona.

Tipo	Artefactos
Perfiles (12)	PacienteLabES, PeticionLab, EspecimenLab, ResultadoLab, InformeLab, LaboratorioOrg, FacultativoLab, CoberturaLab, NotificacionEDO, ProcedenciaValidacion, CohorteVigilancia, TrazaDeAcceso
Extensión propia (1)	codigo-ine, sobre Address
CodeSystem (6)	catálogo-pruebas, enfermedades-edo, resultados-cualitativos, estados-declaracion-edo, modalidades-declaracion-edo, rasgos-de-cohorte

Tipo	Artefactos
ValueSet (10)	pruebas del catálogo, tipos de muestra, motivos de rechazo, catálogo EDO, enfermedades declarables, valores críticos, resultado cualitativo, estados y modalidades de declaración, rasgos de cohorte
ConceptMap (1)	catalogo-a-loinc
Conformidad (3)	SubscriptionTopic del resultado validado, SearchParameter
	notificacion-edo-vencimiento, OperationDefinition de la exportación de cohorte
Ejemplos (38)	Uno o más por perfil; todos validan contra su perfil en CI

Reglas de perfilado que gobernaron el trabajo:

- **Perfilar restringiendo lo mínimo.** Un perfil sobre-restringido no se puede reutilizar.
- **Nada de required sobre conjuntos que en la práctica no están cerrados.** Es un anti-patrón declarado.
- **Un perfil también sirve para prohibir: 0..0 es una regla de verdad.** TrazaDeAcceso cierra AuditEvent.entity.query y entity.detail a 0..0, y no es cosmética: el primero guarda la consulta ejecutada **en base64**, que es donde acabaría el número de historia de un GET [base]/Patient?identifier=... sin que se vea al leer el recurso. Escrita así, la regla vive en el contrato publicado y el validador oficial la comprueba en cada ejemplo. Cuando la regla es «esto no puede viajar», el sitio es el perfil, no un comentario ni un if.
- **Todo ejemplo valida contra su perfil en CI** con el validador oficial. Un recurso que no valida no sale del *pipeline*.
- **En la guía queda escrito** que esto es una simulación con datos sintéticos, que las URIs canónicas son propias y no oficiales, y que ISO 15189 está fuera de alcance.

6.3. Identificadores y nombres españoles

Ésta es la parte del proyecto que más específica es de España, y la que más caro sale si se copia de un ejemplo estadounidense.

El NUHSA y el CIP-SNS no son identificadores paralelos, sino una jerarquía definida por el RD 183/2004 (modificado por el RD 702/2013 y el RD 922/2024):

Código	Qué es	Emisor
CIP-SNS	Código de identificación personal del SNS. Único y vitalicio, actúa de nexo entre los códigos autonómicos	Base de Datos de Población Protegida del SNS
CIP-AUT	El código de identificación personal autonómico	Cada comunidad autónoma
NUHSA	Es el CIP-AUT de Andalucía , no un tipo aparte. Doce caracteres: AN más diez dígitos	Servicio Andaluz de Salud

Modelarlo así —un *slice* de tipo «CIP autonómico» con el **system** de Andalucía— hace el perfil **reutilizable para otra comunidad sin rehacerlo**, y coincide con cómo lo modela el Ministerio.

Identificador	Disponibilidad en un laboratorio privado
NHC propio	Siempre — la Ley 41/2002 obliga a los centros privados a asignar un código único por paciente
DNI / NIE	Casi siempre
NUHSA	A menudo no. Mutualistas y privados con frecuencia no lo conocen
CIP-SNS	Rara vez conocido
NASS	Ocasional
Póliza o número de mutualista	Muy frecuente, y va en Coverage

De ahí la regla **D16**, que es la que hay que respetar al tocar los perfiles:

`system` más `value` como cadena **opaca**, 0..1, Must Support. **Sin pattern ni regex** en los identificadores que el laboratorio no emite. **Solo el NHC propio** lleva 1..1 y validación de formato.

Tres razones, y las tres siguen valiendo: el laboratorio no emite esos códigos y validar su formato solo puede producir falsos rechazos de pacientes reales; sobre-restringir es un anti-patrón declarado; y la estructura **puede cambiar por Real Decreto** —ya se ha modificado tres veces—, con lo que un **pattern** convierte un cambio normativo en un despliegue urgente.

Y el NUHSA nunca es 1..1. En un laboratorio privado, exigirlo sería rechazar a la mitad de los pacientes.

Los dos apellidos. `HumanName.family` lleva el nombre familiar **completo** ("Ojeda Rodríguez"), y dos extensiones estándar lo descomponen: `http://hl7.org/fhir/StructureDefinition/humanname-fathers-family` y `...humanname-mothers-family`. Dos precisiones que cuestan un fallo cada una:

- **El contexto de las dos extensiones es `HumanName.family`, no `HumanName`.** En FSH se declaran sobre el elemento `family`. Equivocarse aquí hace que la guía **no compile**.
- **Nunca partir por el espacio.** "de la Torre Gómez" y "Fernández de Córdoba Ruiz" rompen ese heurístico, y en un laboratorio confundir apellidos es confundir pacientes.

El juego de caracteres. MUÑOZ, ÁLVAREZ y PEÑA son casos de prueba **obligatorios** en todo lo que toque nombres —generador, canales v2, web, app—, no opcionales.

6.4. Las extensiones: solo cuando no existe elemento estándar

La regla se aplicó literalmente, verificando caso por caso contra el paquete canónico, y el resultado fue que **casi todo lo que parecía necesitar una extensión ya tiene elemento estándar en R5**:

Necesidad de negocio	Elemento estándar R5
Rechazo de muestra	<code>Specimen.status</code> (admite <code>unsatisfactory</code>) más <code>Specimen.condition</code>
Prueba refleja	<code>Observation.triggeredBy</code> , con <code>type</code> en <code>reflex</code> / <code>repeat</code> / <code>re-run</code>
Paciente en ayunas	<code>Specimen.collection.fastingStatus[x]</code>
Rango por sexo y edad	<code>Observation.referenceRange.appliesTo</code> y <code>.age</code>
Número de colegiado y titulación	<code>Practitioner.identifier</code> y <code>.qualification</code>
Privado que paga frente a mutua	<code>Coverage.kind</code> , obligatorio en R5
Número de petición que agrupa líneas	<code>ServiceRequest.requisition</code>
Número de acceso de la muestra	<code>Specimen.accessionIdentifier</code>
Quién validó el resultado	<code>Provenance</code>

Necesidad de negocio	Elemento estándar R5
Notificación EDO	Task

Extensiones estándar usadas: tres (los dos apellidos y data-absent-reason). **Extensiones propias:** una, el código INE de municipio y provincia sobre Address, porque no existe elemento ni extensión estándar. Cualquier extensión propia adicional **debe justificarse por escrito** contra esa tabla antes de crearse.

6.5. R5 no es R4: la tabla que hay que mirar antes de escribir el primer mapeo

Ésta es probablemente la página más útil del documento para quien vaya a tocar el código. Comprobada una a una contra el paquete canónico hl7.fhir.r5.core@5.0.0. **Cualquier ejemplo, tutorial, respuesta de una IA o librería basada en R4 que se copie sin mirar va a fallar aquí.**

Elemento	R4	R5	Impacto
ServiceRequest.code	CodeableConcept	CodeableReference	Cambia el JSON de <i>toda</i> petición
ServiceRequest.reason	reasonCode + reasonReference	reason 0..* CodeableReference	Dos elementos fusionados en uno
Coverage.kind	no existe	1..1 obligatorio (insurance / self-pay / other)	Un Coverage de R4 no valida en R5
Coverage.subscriberId	string	0..* Identifier	Cambio de tipo y de cardinalidad
Observation.triggeredBy	no existe	0..*	Es el gancho de las pruebas reflejas
Observation.bodyStructure	no existe	0..1 Reference	
DiagnosticReport.composition	no existe	0..1 Reference	
Specimen.combined / .role / .feature	no existen	nuevos	
Subscription.criteria	cadena de búsqueda dentro de la suscripción	no existe: el criterio es un SubscriptionTopic aparte	Cambia el modelo entero, no un elemento
Subscription.error	string dentro del recurso	eliminado: SubscriptionStatus.error, por \$status	Buscarlo y no encontrarlo invita a inventarse una extensión
Group.actual (boolean 1..1)	así se llama	eliminado: Group.membership (definitional / conceptual / enumerated)	Un Group de R4 no valida ; description pasa de string a markdown
Organization.telecom / .address	existen	eliminados: contact (ExtendedContactDetail)	Un Organization de R4 no valida en R5
ConceptMap.source[x] / .target[x], element.target.equivalence	así se llaman	sourceScope[x] / targetScope[x] y relationship , con códigos distintos	Un ConceptMap de R4 no valida en R5

Elemento	R4	R5	Impacto
ConceptMap ... dependsOn.property (uri)	así se llama	dependsOn.attribute (code) más Concept- Map.additionalAttribute	Modela «este mapeo solo vale si...». HAPI 8.10 no lo sirve (§11.2)
AuditEvent.type + .subtype (Coding)	así se llaman	category (CodeableConcept 0..*) + code (CodeableConcept 1..1)	Cambia el nombre y el tipo de dato
AuditEvent.outcome (código) + .outcomeDesc	dos elementos	outcome con code (Coding) y detail	Un AuditEvent de R4 no valida en R5
AuditEvent.agent.network (address/type)	elemento con hijos	agent.network[x] (Reference / uri / string)	Y agent.who pasa a 1..1; altId, name y media desaparecen
AuditEvent.entity.type / .lifecycle / .name / .description	existen	eliminados ; source.site pasa de string a Reference(Location)	Nuevos: severity, occurred[x], patient, encounter, authorization
Extensiones	dentro del núcleo	paquete aparte hl7.fhir.uv.extensions	Otra dependencia que declarar

Dos consecuencias prácticas:

- Usa siempre el modelo **org.hl7.fhir.r5** de **HAPI**. Si un import trae ...model.r4..., está mal.
- Coverage.kind obligatorio juega a favor del dominio: **self-pay** (privado que paga) frente a **insurance** (mutua o aseguradora) es **exactamente** la distinción de negocio de un laboratorio privado, y R5 la fuerza a estar presente.

6.6. La terminología

El laboratorio **no es un servidor de terminología**: no publica \$expand ni \$validate-code. Pregunta. El puerto es fhir/terminologia/Terminologia, y su implementación habla \$lookup, \$validate-code y \$translate contra una URL de configuración. **No hay tipo de servidor ni operación propietaria que configurar**: apuntar a Snowstorm o a Ontoserver es cambiar esa línea.

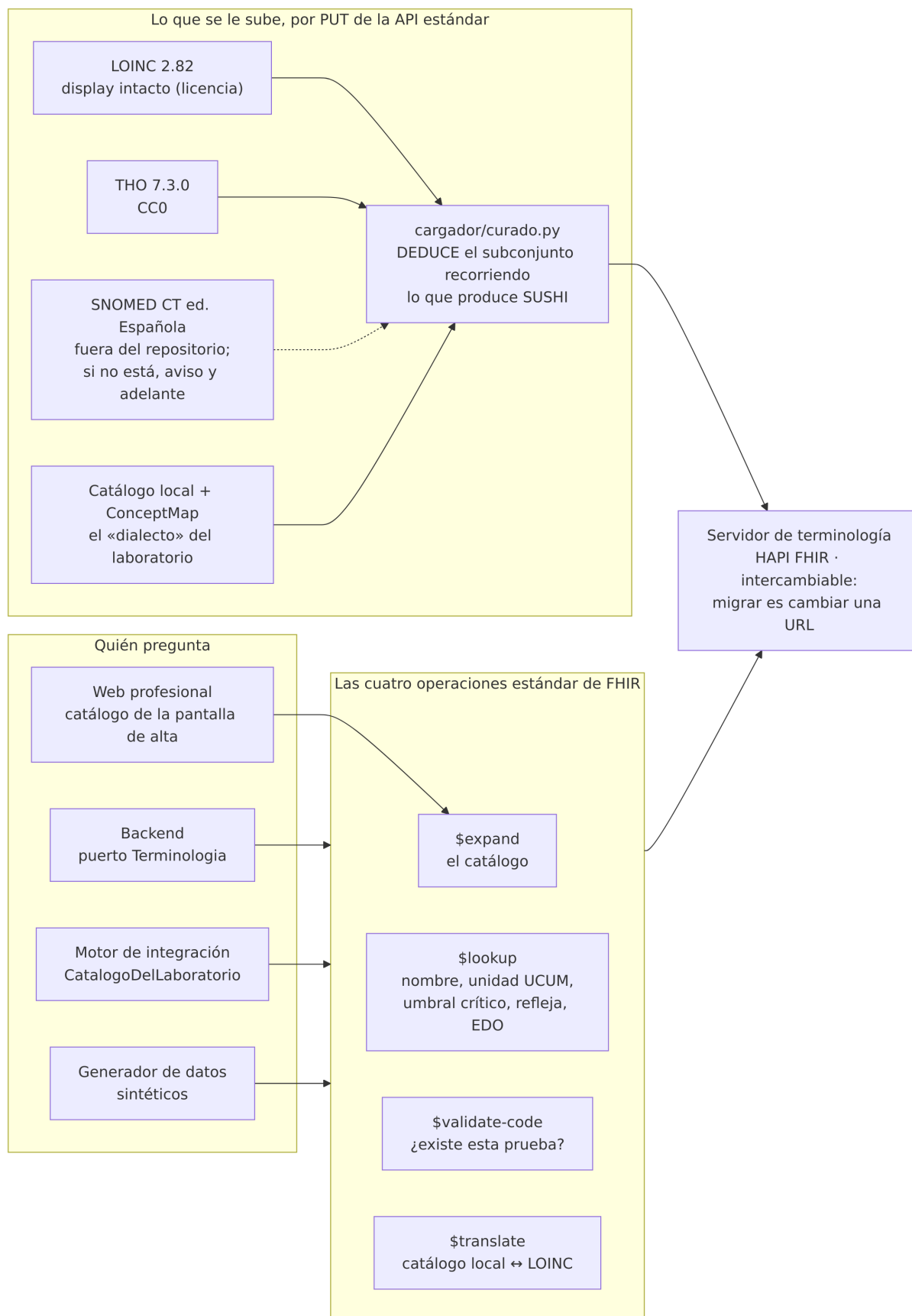


Figura 6.1: Quién pregunta qué al servidor de terminología

- **Nada de `Map<String,String>`.** El puerto no tiene un método que devuelva «el catálogo entero» a propósito: con uno, lo primero que haría alguien es cachearlo al arrancar, y eso es la lista paralela que prohíbe el invariante 4. Se pregunta por un código cada vez.
- **El subconjunto curado se deduce de la guía, no se escribe.** `cargador/curado.py` recorre todos los recursos que produce SUSHI y recoge cada pareja `system/code` que aparezca, a cualquier profundidad. No hay una lista de códigos en ningún sitio: si alguien añade un LOINC a un perfil, el cargador lo sube solo.
- Los subconjuntos se declaran **`content: fragment`**, que es lo que son. Declararlos **`complete`** sería mentir en un elemento que otros servidores usan para decidir si pueden expandir.
- **Se declara siempre la versión exacta del *release* que se carga.** Sin eso los `display` dejan de ser reproducibles: la misma consulta da un nombre distinto contra otro servidor y nadie sabe por qué. La de SNOMED **se deduce del propio *release***, del *refset* de dependencia de módulos, no se escribe a mano.
- **Los `display` de un informe español van en español**, y el nombre del catálogo local manda y va en `CodeableConcept.text`. El `display` de LOINC llega en inglés y **se copia sin tocarlo**, porque su licencia prohíbe alterar el contenido de los campos (`adr-0009`).
- **Solo se publica el LOINC declarado *equivalent*.** Donde el `ConceptMap` dice `source-is-broader-than-target`, publicarlo como si fuera lo mismo afirmaría un método que el laboratorio no ha declarado. La vuelta del mapa se invierte por el mismo motivo solo donde hay equivalencia.
- **Si el servidor de terminología no está, el laboratorio sigue:** código sin nombre y validación que no rechaza, con un aviso por cada caída. El nombre es presentación, el código es el dato — un servidor caído no puede impedir que se registre un resultado.

Del lado del motor v2, lo que se pregunta son tres campos: `OBR-4` —la prueba que pide el volante—, `OBX-3` —la que informa el analizador— y `SPM-4` —el tipo de muestra—. Los tres pasan por `CatalogoDelLaboratorio`, que resuelve contra el mismo servidor con las cuatro operaciones estándar: `$lookup` para el nombre y la unidad UCUM, `$validate-code` para saber si la prueba o el tipo de muestra existen, `$translate` para el LOINC y su vuelta, y `$expand` para contar el catálogo al arrancar. **Ninguna tabla de códigos dentro del motor**, y lo que no tiene equivalencia declarada no se inventa: el mensaje va a la bandeja de errores.

Las reglas del laboratorio viven en propiedades del concepto, dentro de `catalogo-pruebas`: el umbral crítico, la prueba refleja y la obligación de declarar. No son `CodeSystem` ni `ConceptMap` aparte, y el motivo es siempre el mismo: un catálogo paralelo con los mismos códigos crea *conceptos distintos con el mismo código*, y un `$lookup` deja de traerlo todo junto.

Las licencias, que deciden qué puede estar en el repositorio:

Fuente	Redistribuable	Qué implica
LOINC 2.82	Sí, si cada copia lleva el aviso de copyright y la versión y no se altera el contenido de ningún campo	El <code>CodeSystem</code> subido lleva copyright y version , y el nombre largo va intacto
THO 7.3.0	Sí (CC0)	Se extraen solo los sistemas que la guía cita
SNOMED CT Edición Española	No. Gratuita previa licencia del Ministerio, sin redistribución	Ni un fichero en el repositorio. Se monta desde fuera; si no está, el cargador avisa en voz alta y sigue

6.7. Los canales HL7 v2

Versión fijada: **HL7 V2.5.1 (2007)** (D12). La decisión se tomó con evidencia medida: frente a V2.5 son el mismo conjunto de 151 segmentos, el mismo de 344 tablas y el mismo MSH de 21

campos, con un diff dominado por ejemplos saneados y erratas. V2.5.1 es V2.5 con las erratas corregidas, así que elegirla no cuesta nada.

Salvedad registrada: la **tabla 0354** tiene contenido distinto entre V2.5 y V2.5.1, y además **se contradice dentro del mismo estándar** —el capítulo 2 frente al apéndice A—. Manda el capítulo 2 y el canal rechaza lo que no cuadre (**adr-0018**).

Entrante (MLLP sobre TLS)	Produce
ADT^A01 / A08	Patient (demografía, altas y correcciones)
OML^O21	ServiceRequest más Specimen
ORU^R01	Observation (el resultado bruto del analizador)

Saliente: ORU^R01 hacia el HIS cuando el informe se valida.

6.7.1. MLLP: una trampa documental de tres capas

Merece quedar escrito porque no está en ninguna otra parte con este detalle:

1. El **apéndice B** («Lower Layer Protocols») de V2.5 y V2.5.1 **está vacío**: una página que dice que el contenido se movió a la guía de implementación.
2. **Ese documento no es una guía de V2: es un estándar de HL7 Version 3** — *HL7 Version 3 Standard: Transport Specification, MLLP, Release 2*. El protocolo que transporta prácticamente todo el tráfico V2 del mundo está publicado bajo el paraguas de V3.
3. **Está retirado desde mayo de 2025**, con el aviso normativo publicado en junio de 2025, **y no hay sustituto designado**.

Consecuencia real: **hoy no existe un estándar HL7 vigente para MLLP**, mientras MLLP sigue siendo el transporte universal de V2 en producción. **Impacto en este código: ninguno** — HAPI HL7v2 implementa el *framing* (0x0B ... 0x1C 0x0D) y nunca se escribe a mano. Lo que falta es fuente citable, no código.

Capítulo 7

El modelo de seguridad

7.1. Son dos capas, y no son intercambiables

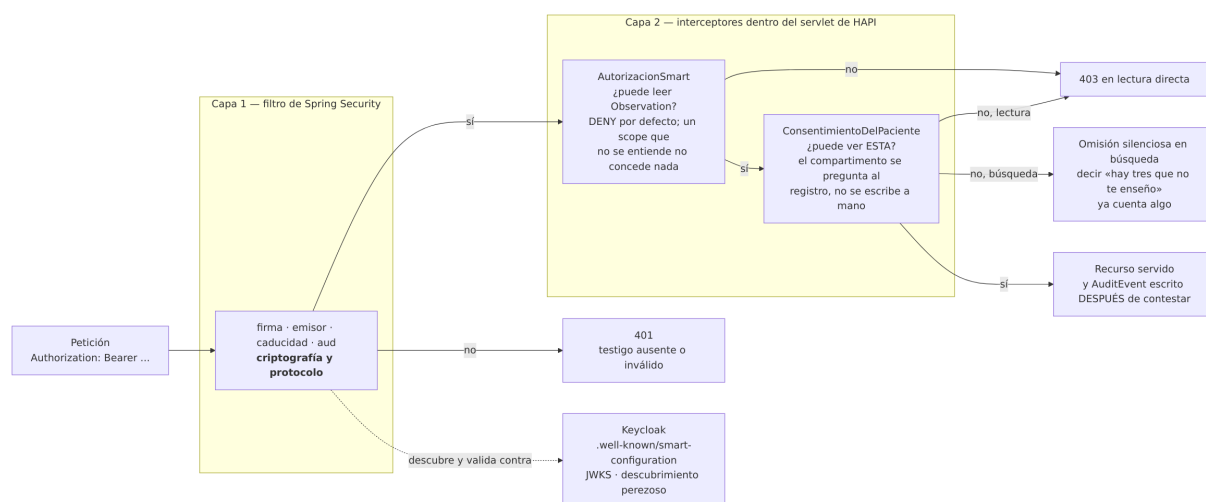


Figura 7.1: Las dos capas: el filtro comprueba el testigo; los interceptores deciden de quién son los datos

La primera capa es un **filtro de Spring Security**: firma, emisor, caducidad y aud. Criptografía y protocolo.

La segunda son **interceptores dentro del servlet de HAPI**: *AutorizacionSmart* decide *si puede leer Observation*, y *ConsentimientoDelPaciente* decide *si puede ver ésta*.

Confundirlas es el error habitual, y por eso conviene decir la consecuencia: **un scope concedido no garantiza los datos**. El consentimiento se aplica en el servidor FHIR, no en el gateway.

7.2. La autorización SMART

AutorizacionSmart traduce los *scopes* a reglas de HAPI, con **PolicyEnum.DENY por defecto**.

- **Un scope que no se entiende no concede nada.** *AmbitoSmart.de()* devuelve vacío ante un sufijo desordenado, repetido o inventado. «Corregir» un *.dus* a *.cud* le daría a quien pidió actualizar el permiso de borrar.
- **aud es obligatorio.** Sin validarlo, un testigo legítimo emitido para otro servidor de recursos del mismo *realm* valdría aquí. Hay un test que lo comprueba.

- **El descubrimiento es perezoso.** El laboratorio arranca con Keycloak caído y contesta 401, que es lo correcto. Lo que no hace es negarse a arrancar.

7.3. El consentimiento: la mitad que no se puede delegar

ConsentimientoDelPaciente decide **de quién son los datos**. El proxy no sabe qué es un compartimento y el proveedor de identidad ya hizo lo suyo al emitir el testigo.

- **El compartimento se pregunta al registro de parámetros de búsqueda** (`getProvidesMembershipInCompartments`), no se escribe a mano.
- **Y vive en un solo sitio.** HAPI sabe hacerlo también con `inCompartment`, y ponerlo en los dos parecería defensa en profundidad: sería la misma regla en dos ficheros que hay que cambiar a la vez.
- **Dos formas de decir que no, y la diferencia importa.** Lectura directa: 403. Búsqueda: se **omite el recurso en silencio**, porque contestar «hay tres que no te enseñé» ya cuenta algo de quien no lo autorizó.

7.4. La exportación masiva: una puerta distinta

`$export` no entrega una historia: entrega **la población entera de una enfermedad** en un fichero que después vive en un disco. Por eso la regla es distinta de la de una lectura (D23):

- **Hacen falta los dos ámbitos a la vez, y el primero por su nombre:** `system/Group.rs` nombrando a `Group` y `system/*.rs`. Un `system/*.rs` a secas *incluye* `Group` y aun así **no basta**: si el comodín valiera, la mitad «autorización sobre la cohorte» de la regla no existiría y cualquier cliente de lectura total exportaría. Un testigo de usuario no exporta ni con `user/*.cruds`.
- **El 403 va antes que el 404:** qué cohortes hay es, en sí mismo, información epidemiológica.
- **La cohorte no la compone nadie: se forma sola al declarar.** El sujeto de un resultado declarado entra en `Group/cohorte-{enfermedad}` en la misma transacción que abre la declaración, y desde fuera el `Group` es de solo lectura. Quien elige a los miembros elige qué se lleva.
- **Lo que sale va seudonimizado**, y es una divergencia consciente del estándar: sexo, año de nacimiento y municipio INE. El exportador **no filtra campos: construye un Patient nuevo** desde una lista blanca, que es la diferencia entre olvidarse de quitar algo y no tener por dónde colarlo.
- **Un parámetro no soportado se rechaza con 400**, al revés que en la búsqueda normal: ignorar un `_since` devolvería la cohorte entera a quien pidió solo lo nuevo, sin decírselo.
- **Nada de PHI en la URL ni en el nombre del fichero.** El sondeo va por el identificador del trabajo y la descarga por un **billete opaco**; en el disco, una carpeta por trabajo.
- **Caduca, un barrendero lo borra, y un DELETE lo borra en el acto.** El barrendero busca además **huérfanos**: ficheros en disco sin trabajo que los reclame, que es lo que deja un reinicio a mitad de exportación.

7.5. Los clientes y sus credenciales

Cliente	Lanzamiento	Tipo	Ámbitos
Web profesional	SMART EHR launch	Público, PKCE S256	<code>user/*.rs</code> más <code>user/Patient.c</code> , <code>user/Practitioner.c</code> , <code>user/ServiceRequest.c</code>
App del ciudadano	SMART standalone launch	Público, PKCE S256	<code>patient/*.rs</code>

Cliente	Lanzamiento	Tipo	Ámbitos
Motor de integración	SMART Backend Services	Confidencial, private_key_jwt	Los cinco system/ que escriben los canales
Exportación analítica	Backend Services, temporal	Creado y borrado por un guion	system/Group.rs y system/*.rs
Reconciliación	—	Sin cliente asignado	system/*.cruds , definido y no concedido

Los clientes públicos no llevan secreto, y no puede ser de otra manera: todo lo que viaja en el paquete que se descarga el navegador lo puede leer cualquiera. Cinco cosas del lanzamiento que no son opcionales:

- **Nada se cablea.** De la base FHIR sale `.well-known/smart-configuration`, y de ahí el `authorization_endpoint` y el `token_endpoint`.
- **El iss se comprueba contra una lista.** Llega por la URL y decide a dónde se manda al usuario a identificarse: aceptar cualquiera es **la** vulnerabilidad clásica del EHR launch.
- **state de 256 bits y PKCE S256**, los dos con `crypto.getRandomValues`. Si el servidor no ofrece S256, no se lanza: caer a **plain** es mandar el verificador en claro.
- **user/*.rs no basta para el alta.** `.rs` es solo lectura y la pantalla de alta **crea** recursos, así que se piden tres permisos de creación y **ni uno más**: `user/*.cruds` daría de paso permiso para borrar informes.
- **El testigo va en un interceptor y solo a las llamadas del laboratorio.** Un testigo enviado a quien no le corresponde es un testigo entregado.

La sesión de la web vive en `sessionStorage` y no en una cookie: la cookie la manda el navegador sola en cada petición al origen, y eso es CSRF; aquí el testigo lo pone la aplicación a mano. Con XSS el testigo es legible y una cookie `httpOnly` tampoco lo salvaría — la respuesta a XSS es que no haya XSS. Y **la guarda de ruta no es control de acceso**: eso se aplica en el laboratorio; la guarda solo evita una pantalla que iba a contestar 401 en cuanto pidiera algo.

El motor de integración se autentica sin usuario, sin navegador y **sin secreto compartido**: firma con su clave privada una aserción de cliente y la canjea por un testigo `system/`. Cuatro cosas de la norma que se incumplen con facilidad y que aquí están respetadas: la aserción dura **cinco minutos como mucho**, el `jti` es **nuevo en cada una**, el `aud` es el `token_endpoint` (no el laboratorio) y **no hay testigo de refresco** — cuando caduca, se firma otra. La clave privada llega por variable de entorno y nunca está en el repositorio; **el JWKS se publica en GET /motor/jwks.json** y Keycloak se lo baja de ahí, de modo que rotar la clave no exige una ventana de indisponibilidad.

Y el arnés con el que se prueba todo eso **verifica la aserción de verdad**: `IdentidadDePrueba` se baja el JWKS del motor por HTTP y comprueba la firma RS384. Un doble que devolviera el testigo sin mirar dejaría sin probar justo la parte que cuesta acertar.

7.6. La auditoría: todo se registra, y sin PHI

Cada lectura y cada escritura de la API escriben un `AuditEvent`: **quién, qué, cuándo y desde dónde, ni una palabra más**.

- **La traza se escribe DESPUÉS de contestar.** Por lo mismo que el notificador EDO no bloquea la validación. Y se escribe con una petición de sistema, porque si no, un testigo de solo lectura no dejaría rastro: el acceso que más interesa registrar sería el único sin registrar.

- **En una traza, la referencia es literal SOLO si la ha publicado este servidor.** HAPI comprueba la integridad referencial al escribir, así que apuntar a lo que no existe **tumba la traza entera** — y la del acceso fallido es la que más falta hace. Va literal lo que salió por la respuesta; van lógicas (`type` más `identifier`) las entidades de lo que solo se pidió, **`agent.who` siempre** —el `fhirUser` lo afirma el proveedor de identidad, no este servidor— y `source.observer`.
 - **Y una traza tampoco puede impedir borrar lo que observó.** Por eso la integridad referencial al borrar está desactivada **solo** para esos caminos: si no, el derecho de supresión sería inejercitable en cuanto alguien hubiera mirado el recurso (`adr-0030`).
 - **`entity.query` y `entity.detail` están prohibidos en el perfil**, a 0..0 (§6.2).
 - **Se busca con POST `[tipo]/_search`, no con GET `[tipo]?...`**, y no por gusto: los criterios llevan el número de historia, y una URL con eso dentro se queda en la barra del navegador, en su historial, en el log del proxy y en la traza del servidor. FHIR admite los mismos criterios en el cuerpo (`adr-0016`).
-

Capítulo 8

Cómo se levanta y cómo se recorre

8.1. Requisitos

Docker y Docker Compose; **JDK 21**; **Node 24**; **Python 3.11 o superior**; Flutter solo para app-ciudadano/. Las comprobaciones se escriben con curl y jq, y los guiones de infra/ usan además bash, python3 y openssl. **Maven no hace falta instalarlo**: el *wrapper* (./mvnw) va en modo only-script, de forma que el repositorio no contiene ningún binario.

Dos cosas viven **fuera** del repositorio y hay que tenerlas antes de levantar: la **guía compilada** (npx --yes fsh-sushi . dentro de ig/, que no se versiona y de la que sale el catálogo de pruebas) y las **releases de terminología** (§13.1).

8.2. Levantar la pila

```
git clone https://github.com/A0jeda006/HispaLIS.git
cd HispaLIS
cd ig && npx --yes fsh-sushi . && cd ..           # la guía compilada
cp infra/compose/.env.example infra/compose/.env # y pon las dos contraseñas
docker compose -f infra/compose/docker-compose.yml up -d
```

Servicio	Dónde
Web profesional	http://localhost:4200
API FHIR	http://localhost:8080/fhir (y en http://localhost:4200/fhir, mismo origen y sin CORS)
Servidor de terminología	http://localhost:8086/fhir
Identidad (Keycloak)	http://localhost:8081, realm hispalis
Kafka y registro de esquemas	localhost:29092 y http://localhost:8085
Motor de integración	localhost:2575 (MLLP sobre TLS). Su consola no se publica

Dos comprobaciones que dicen mucho en una línea:

```
curl -s http://localhost:8080/fhir/metadata | jq '.fhirVersion' # -> "5.0.0"
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/fhir/Patient # -> 401
```

La terminología condiciona el arranque del laboratorio; el bus no. Lo que se espera no es a que el servidor de terminología conteste, sino a que **el cargador termine**: un servidor levantado y vacío responde «no» a todo \$validate-code, que es la peor forma de estar disponible.

Con Kafka parado, en cambio, el laboratorio sigue aceptando escrituras: el hecho queda apuntado en el **outbox** dentro de la misma transacción y el relay lo publica cuando el broker vuelve.

Antes de que el HIS pueda mandar una petición hay que **sembrar el directorio de facultativos** (`infra/fhir/sembrar-facultativos.sh`). Es un guion y no un permiso más del motor **a propósito**: dejarle crear facultativos lo convertiría en autoridad sobre un directorio que solo conoce de oídas, y un número de colegiado mal tecleado en el HIS crearía un facultativo fantasma.

Para empezar de cero:

```
docker compose -f infra/compose/docker-compose.yml --profile '*' down -v
```

El comodín de perfiles **no es opcional**: sin él, `down` deja en pie los contenedores de **receptor** y **svea**, que quedan apuntando a una red que ya no existe.

8.3. El circuito, de extremo a extremo

Esto es lo que el sistema hace, recorrido contra la pila levantada y **con la seguridad puesta**, con dos pacientes sintéticos cuyos nombres son casos de prueba de charset: MUÑOZ DE LA TORRE, Begoña María y PEÑA ÁLVAREZ, Íñigo.

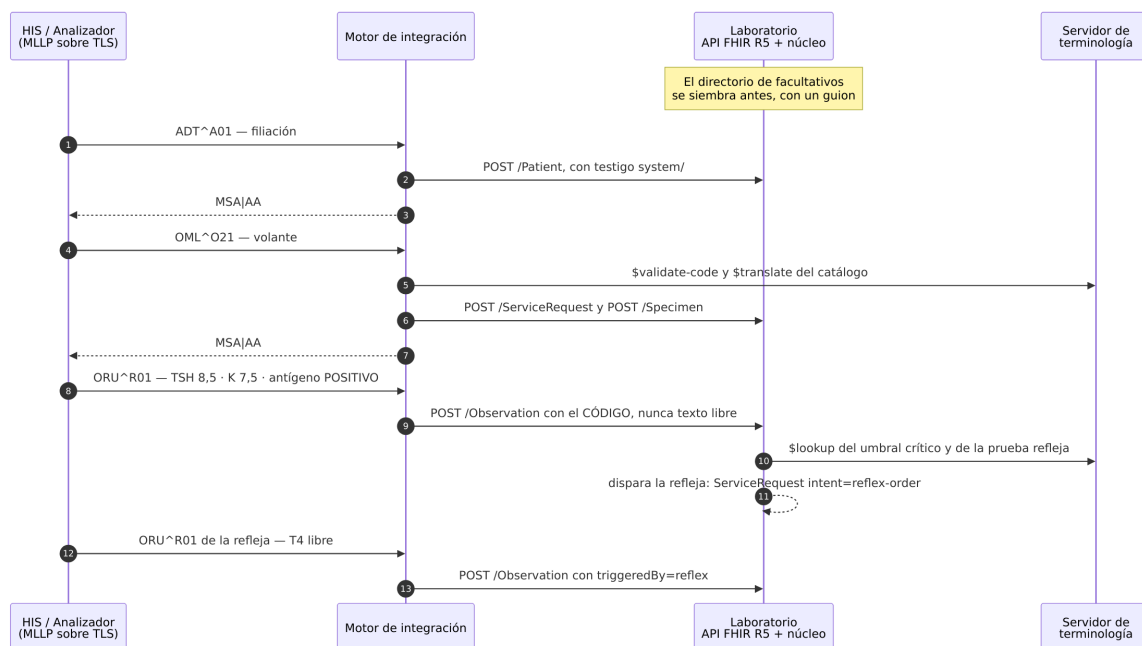


Figura 8.1: Del volante del HIS al resultado, con su prueba refleja

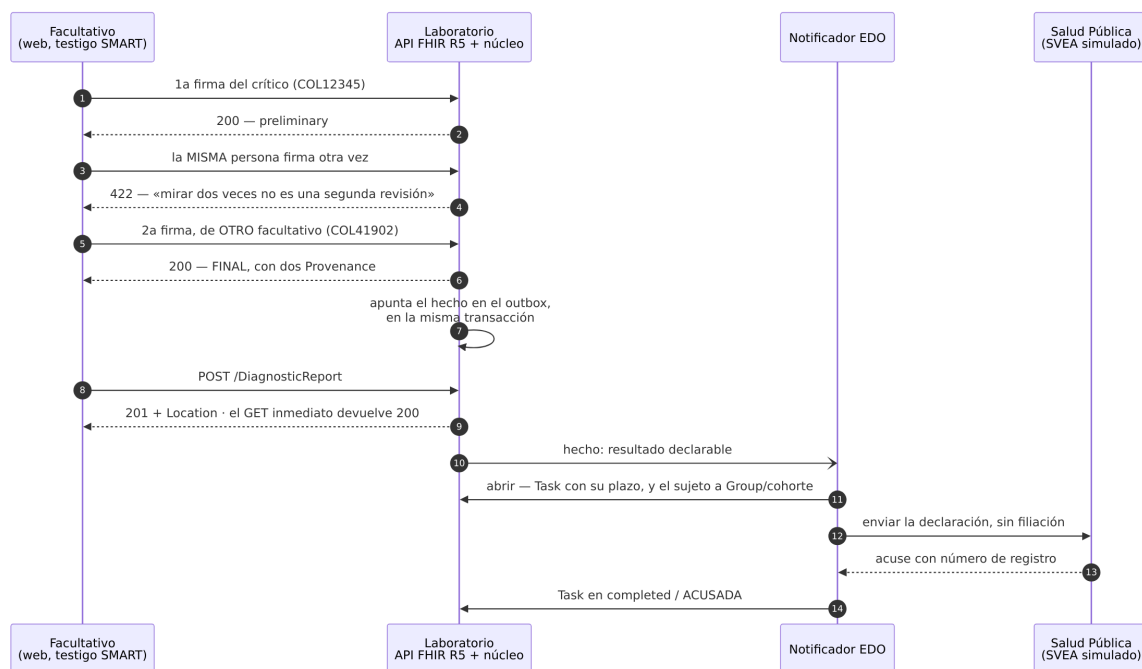


Figura 8.2: De la doble validación del crítico a la declaración acusada

Paso	Qué ocurre
1	Siembra del directorio. Practitioner/COL12345 y COL41902, creados con un testigo obtenido por PKCE
2	El HIS manda por MLLP un ADT^A01 y un OML^O21. Dos acuses MSA AA: filiación y volante
3	El analizador manda un ORU^R01. TSH 8,5; potasio 7,5 mmol/L (crítico); antígeno de <i>Legionella</i> positivo codificado
4	La refleja se dispara sola: un ServiceRequest con intent=reflex-order y, medida, una T4 libre con triggeredBy=reflex. La regla sale del catálogo, no del código
5	Doble validación del crítico. Primera firma: 200, preliminary. La misma persona otra vez: 422, «la misma persona mirando dos veces no es una segunda revisión». Segunda firma de otro facultativo: 200, FINAL, con dos Provenance
6	Informe. 201 más Location, y un GET inmediato a ese Location devuelve 200: read-your-writes
7	Declaración EDO. Un positivo que llega como texto libre no se declara; el mismo positivo codificado sí: Task en completed/ACUSADA, Group/cohorte-legionelosis abierto, y el libro del SVEA con su recuento
8	Traza. Trazas con entity.query relleno: cero. PHI en el volcado de AuditEvent: cero coincidencias
9	\$reconciliar. Sin testigo: 401. Con testigo de profesional: 403, «los scopes no alcanzan»
10	\$export. Se concede el permiso, se exporta, se comprueba sobre el NDJSON descargado que no hay filiación ({gender, birthDate:"1981"} y nada más), DELETE 202, el sondeo pasa a 404 y el cliente temporal se borra del realm

El paso 10 tiene una propiedad que conviene subrayar: **el permiso de exportar se concede, se usa y se retira** en el mismo guion (*infra/fhir/exportar-cohorte.sh*), con un trap que borra el cliente pase lo que pase. Al terminar, el *realm* vuelve a estar como estaba: nadie capaz de exportar.

Cuánto tarda: el circuito clínico entero, del volante al informe, son **2,2 segundos** sobre la pila caliente.

8.4. Construir la guía en local

El IG Publisher se niega a construir si hay un espacio en la ruta del proyecto, y además necesita Jekyll, que en Windows no está. Las dos cosas se resuelven construyendo dentro de la imagen oficial, donde la ruta del contenedor no tiene espacios aunque la del anfitrión sí:

```
docker run --rm --entrypoint bash \
  -v "$PWD/ig":/home/publisher/ig -v ~/.fhir:/home/publisher/.fhir \
  hl7fhir/ig-publisher-base:latest \
  -lc 'cd /home/publisher/ig && java -Xmx4g -jar publisher.jar -ig . -no-sushi'
```

Tarda unos 16 minutos y produce la misma salida que la CI.

8.5. Comandos por componente

Componente	Build	Tests	Lint / formato	Arranque
ig/	<code>npx fsh-sushi .</code> y el IG Publisher	validador oficial sobre lo generado	<code>npx fsh-sushi .</code> con 0 avisos	salida en <code>ig/output/</code>
backend/	<code>./mvnw -q</code> <code>package</code>	<code>./mvnw</code> <code>verify</code>	<code>./mvnw</code> <code>spotless:check /</code> <code>:apply</code>	<code>./mvnw</code> <code>spring-boot:run</code> (o <code>-Parranque-local</code>)
integracion/	<code>./mvnw -q</code> <code>package</code>	<code>./mvnw</code> <code>verify</code>	<code>./mvnw</code> <code>spotless:check /</code> <code>:apply</code>	<code>./mvnw</code> <code>spring-boot:run</code>
web- profesional/ app-ciudadano/	<code>npm run build</code> <code>flutter build</code> <code>web / flutter</code> <code>build apk</code>	<code>npm test</code> <code>flutter test</code>	<code>npm run lint / npm</code> <code>run format</code> <code>flutter analyze</code>	<code>npm start</code> <code>flutter run -d</code> <code>chrome --web-port</code> <code>8090</code>
simuladores/	—	<code>pytest</code>	<code>ruff check . y</code> <code>ruff format</code> <code>--check .</code>	<code>python -m</code> <code>generador --seed</code> <code>42</code>
terminologia/	<code>docker build -f</code> <code>terminolo-</code> <code>gia/Dockerfile</code> <code>.</code>	<code>pytest</code>	<code>ruff check . y</code> <code>ruff format</code> <code>--check .</code>	el servicio terminologia del compose

Cuatro notas de cadena de construcción que sorprenden si no se saben:

- `./mvnw verify` ya comprueba el formato. Spotless con `palantir-java-format` está enganchado a la fase `verify`, así que no hay una orden de *lint* aparte que haya que acordarse de ejecutar.
- `./mvnw spring-boot:run -Parranque-local` levanta su propio PostgreSQL —el mismo binario embebido que usan los tests, sin Docker— y **deja la API abierta, sin testigo**: no hay ningún Keycloak al que preguntar. El arranque lo avisa en el log. Para ejercitar la seguridad está el compose.
- Los tests de Angular corren con `vitest` sobre `jsdom`, el ejecutor por defecto de Angular 22. No es Karma, y `--browsers=ChromeHeadless` no es una opción válida.
- `npm start` y `npm run build` traen antes el catálogo de `ig/fsh-generated/`, así que hay que haber ejecutado SUSHI al menos una vez.

8.6. Integración continua

Siete workflows en `.github/workflows/`, uno por componente, **todos filtrados por paths**: — obligatorio en un monorepo de cuatro *toolchains*, o cada cambio en Flutter recompilaría el backend. La guía se valida con el validador oficial de HL7 contra `hl7.fhir.r5.core@5.0.0` y se publica a GitHub Pages desde `ig/output/`. La app **se empaqueta** en su workflow: analizar y pasar tests no compila la aplicación, así que un fallo del manifiesto no lo veía nadie.

El filtrado por rutas cambia cómo se lee el estado de la CI. Un commit solo dispara los workflows de lo que tocó, así que «los siete en verde» no es una propiedad de un commit sino de cada componente en el último commit que le afectó. Quien los quiera todos a la vez tiene que lanzarlos con `workflow_dispatch`. No es un pendiente: es una propiedad del montaje, y leerla al revés —«hay uno que no ha corrido»— es el error que conviene evitar.

Capítulo 9

Los números finales

Medidos el 16 de agosto de 2026 **desde un clon limpio**, en un directorio que no existía, siguiendo solo el `README.md`, con Docker dentro de WSL2 y sin reutilizar nada: ni `target/`, ni `node_modules/`, ni la caché de Maven del usuario.

9.1. Qué se validó, y con qué

Suite	Resultado
Validador oficial de HL7 sobre el circuito del backend	30 s, sin un solo error
Validador oficial sobre el canal del motor	28 s, sin un solo error
Validador oficial sobre el corpus del generador (644 recursos)	62 s, sin un solo error
Validador oficial sobre los 68 artefactos de la guía qa.html del IG Publisher	49 s, sin un solo error 1 error, 91 avisos, 822 enlaces rotos — que es la línea base documentada , no una regresión (§11.1)

Los únicos avisos que aparecen son `dom-6` —la recomendación de incluir narrativa— y el de canónicas sin fijar.

9.2. Los tests

965 tests, todos en verde:

Componente	Tests	Cobertura
backend/	301	90,3 %
integracion/	299	88,4 %
simuladores/	143	82 %
web-profesional/	98	75,8 %
app-ciudadano/	77	83,8 %
terminologia/ (cargador)	47	92 %

Las tres reglas de testing que la convención exige, comprobadas por lo que corrió y no por lo que existe:

Regla	Quién la cumple	Resultado
<i>Fuzzing</i> de lo que parsea formato externo	FuzzingDelParserTest , por socket, TLS y MLLP escrito a mano	112 tests , 0 fallos
<i>Property-based</i> sobre los invariantes	Cuatro suites: charset en el hilo, ORU saliente, clave de deduplicación, idempotencia	94 tests , 0 fallos
Cobertura medida y leída por los ceros	Los seis componentes, sin umbral y sin check	6 informes, 0 ceros nuevos

Esas dos primeras filas son, además, la explicación de que **integracion/** salte de 86 a 299 tests: 206 de ellos son esas suites.

Y una prueba intermitente es un bug, no un contratiempo. El último rojo del proyecto fue uno: **ReconciliacionDelLaboratorioEnteroTest** encontraba, una ejecución de CI de cada tantas, un **DiagnosticReport** divergente que nadie había tocado. Se cerró **sin tocar el test** — el fallo era del sistema y el test estaba haciendo su trabajo— y con las dos mitades que pide cerrar un intermitente: la causa, en **adr-0045**, y **la próxima vez que caiga, un mensaje que se diagnostica solo**, diciendo de cada referencia inesperada si es del corpus del propio test, qué clase de divergencia es y en qué capa existe. Cuatro tests nuevos lo fijan, y el que lo reproduce lo hace **provocando la condición a mano**, porque con el reloj de verdad es un dado de dos mil caras.

La cobertura se lee por los ceros redondos, no por el porcentaje. La doctrina está en **adr-0041**: un cero redondo admite cuatro veredictos escritos —regla duplicada, regla redundante, camino real sin test, o inalcanzable por diseño— y hay que elegir uno. Los dos huecos que quedan aceptados y escritos son los **__main__.py** del receptor y del SVEA: su lógica vive en el **__init__.py** y está probada; lo que no toca nadie es el **argparse** y el bucle HTTP, que solo corren como contenedor.

9.3. Los apellidos y la ñe, que son un caso de prueba y no un detalle

MUÑOZ, ÁLVAREZ y PEÑA atraviesan el sistema entero —generador, **MSH-18** del canal v2, API, proyección, web y app— con el charset intacto. Los apellidos compuestos del corpus incluyen deliberadamente "de la Torre Gómez" y "Fernández de Córdoba Ruiz", que son los que rompen el heurístico de partir por el espacio.

Una cuarta forma que no es castellana y que un laboratorio de Sevilla ve igual: la **cedilla** —**GONÇALVES** como lo teclea el mostrador en mayúsculas, **Gonçalves** como lo escribe su dueño—. No está entre los tres casos obligatorios —conviene decirlo en vez de dar a entender que sí—, pero el camino es el mismo: todo va en UTF-8 de punta a punta y el juego de caracteres del canal v2 se declara en **MSH-18** en lugar de suponerse. La cadena que compone este mismo documento se comprobó contra los cuatro.

9.4. Cuánto tarda cada cosa

Quien reciba esto necesita saber si verificarlo son dos minutos o cuarenta.

Paso	En frío	En caliente
git clone (689 ficheros)	~5 s	—
npx --yes fsh-sushi .	41 s (0 errores, 0 avisos)	—

Paso	En frío	En caliente
<code>docker compose up -d</code> (construye backend, motor y web)	7 min 47 s	1 min 39 s
Sembrar el directorio de facultativos	3 s	—
El circuito entero, del volante al informe	—	2,2 s
Exportar la cohorte	2 s	—

Desglose del circuito: ADT 0,3 s; OML 0,3 s; ORU 0,5 s; ORU de la refleja 0,4 s; testigo SMART 0,2 s; doble firma 0,2 s; informe 0,1 s; GET inmediato 0,0 s.

Las siete puertas de CI, ejecutadas en local con la misma orden que ejecuta cada workflow:

Puerta	Tiempo	Resultado
ci-backend	954 s en frío, 363 s con caché	301 tests, 0 fallos; Spotless limpio sobre 221 ficheros
ci-integracion	140 s	299 tests, 0 fallos, 4 omitidos
ci-web-profesional	167 s	lint limpio, 98 tests, empaquetado
ci-app-ciudadano	1 955 s (32,6 min)	sin avisos, 77 tests, web y APK de 48,2 MB
ci-simuladores	19 s más 16 s el corpus	ruff limpio, 143 tests, corpus de 644 recursos
ci-terminologia	18 s	ruff limpio, 47 tests, imagen del cargador construida
ci-ig	SUSHI 41 s más IG Publisher 950 s	12 de 12 perfiles con ejemplo, 2 de 2 copias de conformidad idénticas, 68 recursos validados

Y las dos descargas que comparten cuatro workflows: el validador, 187 MB en 15 s; el IG Publisher, 231 MB en 22 s.

Una verificación completa desde cero cuesta poco más de una hora en un equipo de sobremesa de 16 GB. De esa hora, **la mitad es el APK y la guía**. El circuito clínico, que es lo que el sistema hace, son dos segundos.

9.5. El estado de los siete workflows

Al cierre del proyecto los **siete están en verde sobre el mismo commit**, ee52965, lanzados a mano con `workflow_dispatch` precisamente para tener la foto completa:

Workflow	Ejecución
CI · backend	#32
CI · integracion	#12
CI · IG	#13, con sus dos jobs, incluida la publicación en GitHub Pages
CI · web-profesional	#12
CI · app-ciudadano	#8, con el empaquetado dentro
CI · simuladores	#15
CI · terminologia	#10

Capítulo 10

Las decisiones

10.1. D1 a D23

Las veinte primeras se tomaron en el diseño; las tres últimas, construyendo. Cada una tiene su desarrollo completo en `docs/disenio.md` (D1 a D20) o quedó documentada en el plan de trabajo (D21 a D23), y aquí está lo esencial de las tres últimas porque su documento original desaparece.

#	Decisión	Por qué
D1	FHIR R5 (5.0.0)	Una simulación es el caso legítimo de R5; el coste —sin US Core ni IPS, sin Synthea, sin servidores públicos de prueba— está contabilizado
D2	Laboratorio clínico de clínica privada	Único dominio que puntúa alto a la vez en acotable, riqueza FHIR, v2, clientes variados y terminología real
D3	Dominio propio y HAPI como proyección	FHIR es formato de borde: persistir recursos FHIR como entidades pierde los invariantes del negocio
D4	FHIR por aplicaciones, v2 por sistemas	Un navegador no habla MLLP; mezclar los dos contratos en una puerta hace el mapeo inauditable
D5	El motor escribe contra la propia API FHIR	Un solo camino de escritura, con las mismas validaciones, invariantes y auditoría que cualquier cliente
D6	Sevilla, privado puro sin concierto	Fija el marco legal aplicable: EDO sí, ENS no, NUHSA a menudo ausente
D7	SNOMED CT ed. Española y del SNS, LOINC, UCUM	Los <code>display</code> en inglés en un informe español son un error de producto, no un detalle
D8	EDO a Salud Pública; sin Diraya	El contrato del MPA no es público: simularlo da falso realismo. La vía EDO es real, obligatoria y documentada
D9	Tres extensiones estándar y una propia	Verificado contra el paquete canónico: casi todo lo que parecía necesitar extensión ya tiene elemento estándar en R5
D10	Monorepo único con la guía dentro	El contrato es compartido y cambia a la vez: un <code>commit</code> atómico que pasa o rompe la CI de golpe, en vez de un baile de versiones a tres bandas
D11	Motor Spring propio con HAPI HL7v2	Mismo <i>toolchain</i> que el backend y canales como código por construcción; lo que ahorra Mirth no es lo que enseña
D12	HL7 V2.5.1 (2007)	Diff medido : mismos 151 segmentos y 344 tablas que V2.5, dominado por erratas corregidas. Elegirla no cuesta nada
D13	Flutter para la app del ciudadano	El objetivo son clientes multiplataforma y en España iOS es aproximadamente la mitad del mercado
D14	HAPI con subconjuntos curados	La lección es el <i>binding</i> y el contrato de las cuatro operaciones, no operar Snowstorm; y el servidor queda intercambiable cambiando una URL
D15	Generador propio en Python	Synthea apilaría dos problemas difíciles (R4 a R5 y relocalizar a España), y lo difícil son los resultados verosímiles, no la demografía

#	Decisión	Por qué
D16	Sin pattern en identificadores ajenos	El laboratorio no los emite: validar su formato solo produce falsos rechazos, y la estructura cambia por Real Decreto
D17	ISO 15189 fuera de alcance	Es acreditación voluntaria ; la obligación real es la autorización sanitaria del Decreto 69/2008
D18	Nombre: HispaLIS	<i>Hispalis</i> (la Sevilla romana) más LIS ; ancla el proyecto sin ser un topónimo obvio
D19	URIs canónicas propias	Bajo la base https://aojeda006.github.io/HispaLIS/fhir . España no tiene juego oficial consolidado: se definen propias, se publican y se documenta que son propias
D20	Un CLAUDE.md raíz más uno por subproyecto	Un solo fichero con todos los imports sería enorme. (<i>Decisión del entorno de desarrollo; se retira al cerrar el proyecto</i>)
D21	Dos system adoptados del Ministerio, seis propios	Consultado el paquete de definiciones de la guía española de ÚNICAS (que resulta ser también R5): no publica NamingSystem de identificadores, pero usa de facto urn:oid:1.3.6.1.4.1.19126.3 para el DNI y urn:oid:2.16.724.4.40 para el CIP-SNS. Se adoptan esos dos —coincidir con la autoridad nacional no cuesta nada, y lo contrario sería inventar para el DNI una URI que contradice al Ministerio— y se mantienen propios los otros seis, que ÚNICAS no define. Ganancia no prevista: su ValueSet de tipos de documento da códigos SNOMED oficiales del SNS para tipar los <i>slices</i> , y confirma que modelar el NUHSA como CIP autonómico coincide con el criterio del Ministerio
D22	La puerta transaccional sigue cerrada; la atomicidad la pone el reproceso	adr-0014 cerró el procesador de Bundle transaction de HAPI porque llama a las DAO sin pasar por el núcleo. Pero un OML~021 produce ServiceRequest más Specimen, un par que quiere escribirse junto. De tres salidas —abrir la puerta desviándola al núcleo, escribir recurso a recurso con reproceso idempotente, o inventar una operación propia— se eligió la segunda : el reproceso hay que construirlo igualmente, así que no añade trabajo, y la ventana en la que puede quedar un volante sin muestra es exactamente el fallo que la DLQ existe para cerrar . Lo que no se negocia en ninguna de las tres: la puerta no se abre sin que lo que entre por ella pase por el núcleo
D23	Quién exporta, sobre qué, qué sale y qué pasa con el fichero	Cuatro caras de una misma decisión, desarrolladas en §7.4: los dos ámbitos a la vez con Group por su nombre; la cohorte EDO que abre el laboratorio y que de fuera es de solo lectura; el NDJSON seudonimizado construido desde una lista blanca; y las tres formas de que el fichero desaparezca

10.2. Los cuarenta y cinco ADR

Están en docs/adr/, un fichero por decisión, y **no se borran**. Cada uno lleva contexto, alternativas descartadas y consecuencias; aquí va una línea de cada uno para poder localizarlo.

Fichero en docs/adr/	Qué dice
adr-0001-fhir-r5-frente-a-r4.md	Por qué R5 y qué se paga por elegirlo
adr-0002-dominio-propio-y-proyeccion-hapi.md	Dominio propio con proyección HAPI en la misma transacción
adr-0003-identificadores-espanoles-sin-pattern.md	La jerarquía de identificadores, sin pattern , y las URIs canónicas propias
adr-0004-monorepo-con-la-ig-dentro.md	Monorepo único con la guía dentro

Fichero en docs/adr/	Qué dice
adr-0005-motor-propio-con-hapi-hl7v2.md	Motor propio con HAPI HL7v2 frente a Mirth Connect
adr-0006-servidor-de-terminologia-ligero-e-intercambiable.md	Servidor de terminología ligero e intercambiable, no Snowstorm de entrada
adr-0007-trampas-del-ig-publisher.md	Las cuatro trampas del IG Publisher que hay que resolver antes de escribir FSH
adr-0008-windows-desarrolla-linux-construye.md	Finales de línea y bit de ejecución: las dos cosas que git no lleva igual
adr-0009-display-de-terminologia-externa.md	No se fija a mano el <code>display</code> de un código de terminología externa
adr-0010-el-idioma-de-una-ig-se-declara-o-se-assume-ingles.md	El idioma de una guía se declara, o el publisher lo asume inglés
adr-0011-empotrar-hapi-jpa-en-una-aplicacion-propia.md	Empotrar el servidor JPA de HAPI, y las siete trampas de hacerlo
adr-0012-una-sola-transaccion-entre-dominio-y-proyeccion.md	Cómo se consigue de verdad una sola transacción entre dominio y proyección
adr-0013-arrancar-en-local-sin-docker-con-el-postgres-de-los-tests.md	Arrancar en local sin Docker, con el PostgreSQL que ya usan los tests
adr-0014-cerrar-todas-las-puertas-de-escritura-del-framework.md	Un framework que también escribe tiene varias puertas, y hay que cerrarlas una a una
adr-0015-los-datos-de-configuracion-no-van-en-las-migraciones.md	Los datos de configuración no van en las migraciones de esquema
adr-0016-los-criterios-de-busqueda-sensibles-van-en-el-cuerpo.md	Un identificador de paciente no viaja en la URL de búsqueda
adr-0017-los-enlaces-los-firma-el-servidor-y-tras-un-proxy-los-firma-mal.md	Los enlaces los firma el servidor, y detrás de un proxy los firma mal
adr-0018-la-tabla-0354-se-contradice-consigo-misma.md	La tabla 0354 se contradice consigo misma: hay que elegir fuente antes de mapear
adr-0019-una-busqueda-cacheada-convierte-la-idempotencia-en-una-ilusion.md	Una búsqueda cacheada convierte la idempotencia en una ilusión
adr-0020-una-regla-de-seguridad-que-no-casa-deja-la-puerta-abierta-en-silencio.md	Una regla de seguridad que no casa deja la puerta abierta en silencio
adr-0021-el-charset-de-un-mensaje-v2-viaja-dentro-del-mensaje.md	El charset de un mensaje v2 viaja dentro del mensaje, y quien no lo lee rompe la ñe
adr-0022-el-tls-de-un-canal-no-se-configura-con-propiedades-de-la-jvm.md	El TLS de un canal no se configura con propiedades de la JVM
adr-0023-con-una-sola-particion-una-clave-de-reparto-mal-elegida-parece-correcta.md	Con una sola partición, una clave de reparto mal elegida parece correcta
adr-0024-el-contexto-de-lanzamiento-de-smart-no-esta-dentro-del-testigo.md	El contexto de lanzamiento de SMART no está dentro del testigo
adr-0025-el-retorno-de-una-autorizacion-no-se-parece-en-movil-y-en-web.md	El retorno de una autorización no se parece en móvil y en web
adr-0026-un-parametro-que-solo-falla-con-el-servidor-recien-cargado.md	<code>count=0</code> en <code>\$expand</code> : un parámetro que solo falla con el servidor recién cargado
adr-0027-una-credencial-dentro-de-un-recurso-que-la-api-sirve-es-una-credencial-publicada.md	Una credencial dentro de un recurso que la API sirve es una credencial publicada
adr-0028-una-regla-condicionada-no-se-publica-donde-el-servidor-no-la-sirve.md	Una regla condicionada no se publica donde el servidor no la sirve
adr-0029-un-searchparameter-en-draft-se-publica-y-no-se-indexa.md	Un <code>SearchParameter</code> en <code>draft</code> se publica, se lee y no se indexa
adr-0030-la-traza-del-acceso-que-falla-es-la-que-el-servidor-se-niega-a-guardar.md	La traza del acceso que falla es la que el servidor se niega a guardar
adr-0031-cazar-una-excepcion-no-deshace-el-rollback-only.md	Cazar la excepción no deshace el <code>rollback-only</code>
adr-0032-el-display-que-el-fsh-no-escribe-y-el-lookup-no-inventa.md	El <code>display</code> que el FSH no escribe y el <code>\$lookup</code> no inventa

Fichero en docs/adr/	Qué dice
adr-0033-autorizar-la-operacion-no-autoriza-la-segunda-vez.md	Autorizar la operación no autoriza la segunda vez que escribe
adr-0034-un-codigo-que-llega-como-frase-deja-de-ser-un-codigo.md	Un código que llega como frase deja de ser un código
adr-0035-montar-un-volumen-no-es-poder-escribir-en-el.md	Montar un volumen no es poder escribir en él
adr-0036-lo-que-el-parser-tira-sin-decir-nada-revienta-lejos.md	Lo que el parser tira sin decir nada revienta lejos
adr-0037-el-camino-que-atende-el-fallo-tambien-falla.md	El camino que atiende el fallo también falla
adr-0038-medir-la-cobertura-cambia-lo-que-se-mide.md	Medir la cobertura cambió lo que se medía
adr-0039-una-edicion-de-snomed-no-se-descarga-se-compone.md	Una edición de SNOMED no se descarga: se compone
adr-0040-un-refset-oficial-se-referencia-no-se-copia.md	Un <i>refset</i> oficial se referencia, no se copia
adr-0041-un-cero-redondo-de-cobertura-suele-ser-una-regla-duplicada.md	Un cero redondo de cobertura suele ser una regla duplicada
adr-0042-una-propiedad-mal-enunciada-da-un-rojo-que-es-del-test.md	Una propiedad mal enunciada da un rojo que es del test
adr-0043-un-env-que-leen-dos-parsers-no-es-un-formato-son-dos.md	Un <i>.env</i> que leen dos parsers no es un formato, son dos
adr-0044-una-cache-montada-no-es-una-cache-usada.md	Una caché montada no es una caché usada
adr-0045-una-marca-de-tiempo-que-no-sobrevive-a-su-almacen.md	Una marca de tiempo que no sobrevive a su almacén divergirá, una de cada dos mil veces

docs/destilacion.md inventaría, ADR por ADR, qué tiene cada uno de transversal más allá de este proyecto y a qué documento de convenciones iría. También se conserva.

Capítulo 11

Trampas medidas: el conocimiento que no es decisión ni número

Todo lo que sigue está **medido contra las versiones concretas que este proyecto usa**. Es la parte que más caro sale volver a descubrir y la que menos aparece en la documentación oficial.

11.1. De la guía y de la cadena FSH

- **SUSHI compila; el validador conforma. No son lo mismo.** SUSHI comprueba la sintaxis FSH y que los perfiles cuadren; **no comprueba las invariantes de los recursos que genera**. El `SearchParameter` de la guía estuvo dos días incumpliendo `spd-1` de R5 con SUSHI diciendo «0 errores, 0 avisos». Lo vio el validador oficial, que es exactamente por lo que la CI lo ejecuta además. Un artefacto de conformidad **no está comprobado** hasta que ha pasado por el validador, y eso incluye los que no son ejemplos.
- **El idioma de una guía se declara, o el publisher asume inglés.** Con `language: es`, `jurisdiction: ES`, `i18n-default-lang: es` y `resource-language-policy: all-ig` puestos en `sushi-config.yaml`. Sin ellos, la guía se publica etiquetada `lang="en"` bajo `/en/` **con todo el build en verde**: ninguna herramienta detecta que el texto está en otro idioma.
- **Un `ValueSet` se ata al `CodeableReference`, no a su `.concept`.** El camino que parece natural —`* reason.concept from MiValueSet (required)`— SUSHI lo **rechaza**. Va sobre el elemento entero; las cardinalidades sí sobre el `.concept`. En R4 el elemento era un `CodeableConcept` y el camino natural funcionaba, así que cualquier ejemplo copiado de allí falla.
- **Sistema#CODIGO en FSH compila a un `Coding` sin `display`, y nada lo avisa.** La forma con nombre lleva el literal detrás. SUSHI compila las dos sin un aviso y el validador oficial da cero errores en las dos, porque un `Coding` sin `display` es perfectamente válido. Costó un fallo que solo se vio en vivo. **La regla que vale para todo el FSH:** de una referencia entre conceptos se lee la **identidad** —sistema y código— y el contenido se le pide al dueño con su propio `$lookup`.
- **El `id` de un `Instance`: sale del nombre del bloque, que es `PascalCase`.** En un artefacto de conformidad escrito como `Instance:` hay que poner `* id = "kebab-case"` explícito, o la página publicada y la URL canónica dirán cosas distintas.
- **Las cuatro trampas del IG Publisher**, ninguna de las cuales da un mensaje que apunte a su causa:
 1. **`ig.ini` se mantiene a mano y está versionado.** SUSHI retiró la propiedad `template` y ya no lo genera. Casi toda la documentación que se encuentra dice lo contrario: está desactualizada.

2. **ig.ini no admite líneas de comentario.** Un ; antes de [IG] hace que el publisher aborte con «unable to find an ig.ini», culpando a la ausencia del fichero, que sí está. **Añadir un comentario «para aclararlo» rompe la construcción.**
3. **La plantilla es fhir2.base.template.** fhir.base.template ya no se considera segura ni está mantenida, y está anunciado que el publisher se negará a ejecutarse con ella.
4. **El publisher necesita Jekyll,** que no viene en el .jar. Sin él, la construcción recorre entera la fase FHIR y muere al renderizar las páginas.
- **El qa.html de la guía publicada es la línea base.** El publisher termina con código de salida 0 aunque su QA cuente errores, así que «cuántos» solo significa algo comparado con la última CI verde. Hoy esa base es **1 error** —un fichero de mensajes suprimidos que la plantilla espera por defecto y esta guía no tiene— y **822 enlaces rotos**, que son las anclas **#terminology** y **#example** de la barra de navegación: la plantilla las genera en inglés y las páginas están en español. Cada artefacto nuevo suma una docena. **No los cuentes como regresión sin mirar la base.**

11.2. De los servidores

HAPI FHIR 8.10, medido:

- **No trae \$status ni \$events de Subscription** (solo \$trigger-subscription). Están implementados aquí en un proveedor **suelto**, no en un **ProveedorPropio**: hacerlo así sustituiría al proveedor de HAPI para Subscription y le aplicaría las puertas de los recursos con agregado detrás, que este no tiene.
- **No implementa dependsOn en \$translate**, ni a la entrada (dependency) ni a la salida. Medido sobre TranslationQuery. Es justo el elemento con el que R5 modelaría «este mapeo solo vale si el resultado es positivo», así que **el criterio de una regla condicionada no se publica en el ConceptMap**: iría a un elemento que el servidor no sirve. Va como propiedad del concepto, con un solo \$lookup (adr-0028).
- **\$translate en R5 manda sourceCode y targetSystem**, no code y targetsystem. HAPI acepta los dos, así que copiar un ejemplo de R4 *funciona aquí* y falla contra un servidor estricto. A la vuelta pasa lo contrario: HAPI aún devuelve match.equivalence de R4, así que hay que leer las dos formas.
- **La vuelta del \$translate con targetCode no está implementada** (HAPI-1154). El cliente pide las dos formas, la de R5 primero, y cae a reverse=true, que es la de R4.
- **Un SearchParameter en draft no se indexa.** El registro se queda solo con los ACTIVE. El recurso se guarda, se publica y se lee — y la búsqueda contesta HAPI-0524: **Unknown search parameter**, sin error y sin aviso. Por eso el de la guía va en active aunque la guía entera esté en draft: el status de un recurso de conformidad habla de la madurez de la definición, no del proyecto; lo que dice que esto es una simulación es experimental = true. Vale para cualquier artefacto que el servidor tenga que obedecer, no solo para éste.
- **Un read de una DAO que lanza dentro de una transacción la marca rollback-only, y cazar la excepción no lo deshace.** Dentro de una transacción, «¿existe esto?» se pregunta buscando (adr-0031).
- **\$expand necesita el índice de texto completo.** Sin Hibernate Search contesta HSEARCH800001; las otras tres operaciones funcionan igual.
- **count=0 no significa lo que dice la norma, y falla de forma intermitente.** La norma lo define como «devuélveme solo el total»; HAPI lo interpreta como «máximo 0 códigos» y aborta con HAPI-0831. Y solo lo hace mientras el ValueSet está sin pre-expandir, porque la expansión en memoria es la que aplica el límite. Pasa siempre un count real (adr-0026).

- **\$lookup de LOINC exige version** cuando hay un CodeSystem de LOINC cargado como fragmento: sin ella responde HAPI-1738.
- **El dialecto tiene que ser el de HAPI** (HapiFhirPostgres94Dialect), o el arranque avisa de que «dialect is not a HAPI FHIR dialect» y sigue con medio servidor.
- **La imagen del servidor de terminología es *distroless*: solo lleva java.** Un health-check con curl no falla por estar el servidor mal, falla por no existir curl, y el servicio se queda «unhealthy» con el log en verde. Quien espera terminología espera al **cargador**, que además es la condición correcta.

Keycloak, medido:

- **No arranca si una descripción de ámbito pasa de 255 caracteres.** CLIENT_SCOPE.DESCRPTION es varchar(255); --import-realm falla y el servidor no levanta. El *realm* queda a medias y hay que down -v.
- **private_key_jwt exige iat.** Sin él contesta invalid_client con «Token expiration is too far in the future and iat claim not present in token»: mide la vida de la aserción desde iat y, a falta de iat, desde un margen suyo mucho más corto que los cinco minutos que permite la norma.

11.3. De la seguridad y del cliente

- **securityMatcher("/fhir/**") no casa, y no avisa.** La API FHIR la sirve el servlet de HAPI, no el DispatcherServlet. Con spring-webmvc en el *classpath*, una cadena en securityMatcher o requestMatchers se convierte en un MvcRequestMatcher que **nunca empareja** esas peticiones: la cadena se construye, el log dice que va a asegurar el patrón, y la **API queda abierta sin un solo error**. Se usan emparejadores explícitos (PathPatternRequestMatcher) y hay un test que pide sin testigo y exige 401 (adr-0020).
- **proxy.conf.json lleva "xfwd": true, y no es decorativo.** El servidor firma Bundle.link[relation=next] con la dirección por la que le llegó la petición; sin las cabeceras X-Forwarded-*, la página siguiente apuntaría a localhost:8080 y el navegador no la alcanzaría. El cliente **no puede corregir esa URL** porque para él es opaca, así que el fallo solo aparecería al pasar de la primera página (adr-0017).
- **Los system de identificador viven en un solo fichero del cliente** (src/app/fhir/sistemas.ts) y un test los cruza contra ig/input/fsh/aliasess.fsh. Añadir uno obliga a añadirlo también al test.
- **Autorizar la operación no autoriza la segunda vez que escribe.** La regla autorizaba create sobre Provenance, y la segunda firma de un crítico las **reescribe**: es un UPDATE. Con la seguridad puesta, un crítico no se podía terminar de validar (adr-0033).

11.4. De la cadena de construcción y del entorno

- **Se desarrolla en Windows y se construye en Linux, y hay dos atributos que git no lleva igual.** Los finales de línea se resuelven con .gitattributes; el **bit de ejecución no lo gobierna .gitattributes**. Todo guion que la CI invoque como ./script tiene que estar en el índice como 100755 (git update-index --chmod=+x). En /mnt/c el sistema de ficheros da 0777 a todo, así que en Windows un guion **siempre** parece ejecutable y el modo real solo está en el índice (adr-0008).
- **Un .env que leen dos parsers no es un formato, son dos.** Docker Compose lo lee como configuración; source en bash lo **ejecuta**, así que un valor con un espacio se convierte en una orden y el guion muere con rc=127 antes de hacer nada. La solución es un lector compartido (infra/entorno.sh), uno y no una copia por guion (adr-0043).
- **Una caché montada no es una caché usada.** Maven no mira \$HOME: el repositorio local

lo saca de `${user.home}`, que la JVM resuelve por el uid contra `/etc/passwd`. Montar `~/m2` en un contenedor cuyo uid 1000 es otro usuario deja las dependencias dentro del contenedor, y el `--rm` se las lleva. Hay que pasar `-Dmaven.repo.local`. El engaño era que el volumen **no estaba vacío**: el guion del *wrapper* sí respeta `$HOME` y dejaba ahí la distribución de Maven (adr-0044).

- **palantir-java-format no funciona con JDK 25.** Si el JDK del equipo es más nuevo que el 21 de la CI, los tests corren y `spotless:check` revienta con un `NoSuchMethodError`. Se comprueba en un contenedor `temurin:21`. **Y ese contenedor necesita `--user "${id -u}:${id -g}"` para pasar los tests:** el PostgreSQL embebido no arranca como `root`, `initdb` se niega, y el informe de fallo no menciona el motivo por ningún lado.
- **La primera vuelta de Maven es lenta por una razón que no es la red.** El `pom` declara el repositorio de Confluent —hace falta, las *serdes* de Avro no están en Central— y un repositorio declarado en el `pom` se consulta **antes** que el heredado, así que cada artefacto del árbol pide primero a Confluent, se lleva un fallo y vuelve a pedir a Central. Se paga una sola vez. Se dejó documentado y **no se tocó el pom**: reordenar repositorios al cierre cambia de dónde sale cada `jar`.
- **WSL2 se queda con la mitad de la RAM.** En un equipo de 16 GB son 7,6 GB, y ahí **no caben a la vez la pila entera y un verify de Maven**: la máquina virtual entra en *thrashing*, el demonio de Docker deja de responder y hay que `wsl --shutdown`. Y **WSL apaga la distro cuando no queda ninguna sesión abierta**, y con ella se van los contenedores.
- **En Windows, clonar en una ruta corta.** La ruta más larga del repositorio son 117 caracteres, así que en una carpeta profunda el clon se pasa del límite de 260 y git deja ficheros sin escribir, uno a uno.
- **Spring cachea los contextos de test, y eso contamina entre clases.** Una clase que declara su propio `@SpringBootTest` oculta el del padre **entero**, propiedades incluidas, y arranca con los valores de producción. Con la seguridad eso da la cara enseguida; con los interruptores que consumen el `outbox`, no: el contexto de la clase descuidada sigue vivo después de terminar, le quita el hecho a otra clase y lo descarta con su propio catálogo. **Lo que se ve, tres ficheros más allá, es una espera agotada sin una sola excepción**, y el orden en que caigan las clases decide si aparece. Pasó **seis veces**. La defensa ya no es documentación sino un test, `InterruptoresDeContextoTest`, que recorre las clases compiladas y exige que las que declaren su propio arranque **mencionen** todos los interruptores del padre — **y la lista se deduce de la anotación del padre**, porque escrita a mano nacería correcta y quedaría vieja el día que el padre gane un interruptor nuevo, que es la forma de esta trampa y no su cura.

11.5. Lo que solo se ve ejecutando

Cuatro rondas de este proyecto consistieron en **ejecutar lo que ya estaba escrito** —la última, en ejecutarlo con volumen y en otro sistema operativo—, y las cuatro encontraron cosas que ninguna lectura y ningún test veían. Merece quedar como método:

- **Recorrer el circuito entero contra la pila levantada y con la seguridad puesta** destapó **siete fallos que 290 tests no veían**, y todos por el mismo motivo: *un test elige su configuración y un sistema montado no*. Entre ellos: el *realm* no definía los ámbitos del último hito, así que suscribirse y exportar era **imposible** contra el `compose`; una *Legionella* positiva **no se declaraba nunca** si entraba por el analizador, porque el motor colapsaba el código en texto libre (adr-0034); **ninguna** suscripción recibía nada, por un `NullPointerException` que reventaba la vuelta entera del relay (adr-0036); `$export` no podía escribir en su volumen (adr-0035); y **nada se podía borrar** si alguien lo había mirado, porque el ajuste de integridad referencial solo lo consulta `$delete-expunge` y no un `DELETE` normal

(adr-0030).

- **Ejecutar el README.md orden por orden** encontró cuatro órdenes que no funcionaban y seis afirmaciones que habían dejado de ser ciertas. La más instructiva: `-Parranque-local` llevaba ocho días inservible porque la seguridad, añadida después, hacía que la API se negara a levantar sin emisor. Nadie volvió a ejecutarlo.
 - **Verificar desde un clon limpio** encontró cinco «funciona en mi máquina» de los que cuatro dependían de estado que el usuario no tiene: los guiones que morían con un `.env` con espacios, la caché de Maven inerte, un `-o` que exigía una caché que nada llenaba y un guion sin bit de ejecución. El quinto solo apareció en la CI y es el del `@SpringBootTest` de arriba.
 - **Ejecutar con volumen y en otro sistema operativo** destapó el último fallo del proyecto, y es el único que **ninguna ejecución local podía encontrar**: un `DiagnosticReport` de cada dos mil salía publicado con una marca de emisión un milisegundo distinta de la que el dominio guardaba, porque `Instant.now()` da nanosegundos, `timestampz` **redondea** a microsegundos y el `instant` de FHIR publica milisegundos. El reconciliador —que compara las dos— lo veía como una divergencia real, y lo era. No se reproduce en Windows porque su reloj no llega a la franja: medido, `Instant.now()` devuelve **17 valores distintos** por debajo del milisegundo en 300 000 muestras, y ninguno en la franja mala. Hicieron falta un barrido de sesenta pacientes y un *runner* Linux para pisarla una vez. Está en `adr-0045`, con las dos lecciones de método que deja: **un mensaje de fallo que no dice de dónde sale lo que sobra** **cuesta ejecuciones enteras** —este rojo se fue dos sin diagnosticar— y **un verde de un commit que no toca el componente no es un verde**, que es como se dio por cerrado la primera vez.
-

Capítulo 12

Qué es demostración y NO vale para producción

Sin suavizar. Esto se monta para enseñar cómo encajan las piezas, **no para poner a nadie detrás**.

12.1. Credenciales y secretos

- **Las contraseñas del compose son de juguete.** `POSTGRES_PASSWORD: hispalis`; un `.env` con dos contraseñas que pone quien clona —y una tercera clave si se levanta el receptor—; **tres usuarios de demostración con la misma contraseña.**
- **El certificado del canal MLLP es autofirmado** y se genera al levantar la pila.
- **El APK está firmado con la clave de depuración** de Android.
- No hay rotación de secretos, ni gestor de secretos, ni separación de entornos.

12.2. Disponibilidad y escala

- **Instancia única en todo lo asíncrono.** El relay del outbox, el notificador EDO, el barrendero de exportaciones y el sondeo de `$export` **dan por hecho que solo hay un proceso**. Dos backends contra la misma base abrirían la misma declaración dos veces; lo impide un `UNIQUE`, así que el peor caso es una transacción que revienta y se reintenta, pero el desplazamiento de `edo.hecho_consumido` **no está diseñado para varios lectores**.
- **Los NDJSON exportados viven en un disco local, no en un almacén de objetos.** El diseño prevé `MinIO` y el `compose` no lo trae. Con una sola instancia no se nota —el que sondea es el mismo proceso que escribió el fichero—, pero es la misma limitación.
- **Del Redis que dibujaba el diseño tampoco hay nada**, y conviene decirlo porque `docs/disenio.md` sigue nombrándolo: iba a servir de caché de terminología y de soporte de la deduplicación, y ninguna de las dos cosas acabó necesítándolo. La primera la prohíbe el invariante 4 —se pregunta código a código, y cachear el catálogo entero es justo la lista paralela que no puede existir—; la segunda vive en `PostgreSQL`, por `hechoId` en los consumidores del bus y por `MSH-10` en el motor.
- **El contador de MSH-10 de los acuses del motor es efímero:** el contenedor no tiene volumen para el fichero de identificadores de `HAPI`.
- No hay alta disponibilidad, ni copias de seguridad, ni plan de recuperación, ni observabilidad más allá de los logs.

12.3. Superficie expuesta

- **La consola del motor de integración no tiene autenticación**, y por eso **no se publica** fuera de la red del compose. Es una decisión consciente, no un olvido: una bandeja de errores con referencias a pacientes no se abre al equipo.
- **`./mvnw spring-boot:run -Parranque-local` deja la API abierta sin testigo**. Es el arranque sin Docker y por tanto sin Keycloak; el propio arranque lo grita en el log.
- **La CI publica la guía en GitHub Pages**. La guía es pública a propósito; el resto del sistema no se despliega en ningún sitio.

12.4. Los terceros son simulados

El HIS, el analizador, el receptor de notificaciones y el servicio de Salud Pública viven en este repositorio y responden como conviene para la demostración. En particular:

- **El formato de la declaración EDO es verosímil, no fiel**. El contrato de Redalerta no es público.
- **La declaración va sin filiación**, al revés que una declaración real, y el SVEA simulado lo **exige** desde el otro lado.
- **El catálogo de enfermedades EDO y los umbrales críticos son un subconjunto ilustrativo**, no la relación oficial completa.

12.5. Y lo que se dice en la propia guía

Que es una **simulación** con datos sintéticos, que las **URIs canónicas son propias y no oficiales**, y que **ISO 15189 está fuera de alcance**. Los recursos de conformidad llevan `experimental = true`, que es el elemento con el que FHIR dice exactamente esto.

Capítulo 13

Qué queda abierto

Lista **cerrada**: esto es todo lo que se sabe que falta. Un proyecto que se cierra diciendo lo que le falta está terminado; uno que lo esconde, no.

13.1. Bloqueado por datos que no se pueden redistribuir

1. SNOMED CT no está cargado. Es el único ítem del plan que quedó sin cerrar, y **no es trabajo pendiente: es una descarga que falta.**

La Edición Española del SNS **no se redistribuye**. La licencia de afiliado es **gratuita** para quien reside y trabaja en España y se pide con un formulario en el Área de Descarga del Ministerio de Sanidad, que es el Centro Nacional de Referencia y el único distribuidor en territorio español; la barrera, por tanto, no es el permiso sino el fichero.

Consecuencia concreta: los **diez** códigos SNOMED que la guía referencia —los diez son tipos de muestra; los resultados cualitativos son un `CodeSystem` propio y deliberado— se publican y **no se pueden validar contra el servidor**. El hueco está modelado, el cargador lo avisa en voz alta al arrancar y el sistema funciona sin él, porque nada obligatorio depende de SNOMED.

Y hay dos cosas que hay que saber antes de intentarlo:

- **No es un fichero, son tres productos a versiones ancladas.** La variable de configuración no apunta a *una* release, sino a una raíz con tres descomprimidas: la **Edición Internacional** (los conceptos), la **Spanish Edition** (solo las descripciones en español) y la **extensión del SNS** (sus propios conceptos y descripciones complementarias). **No valen las últimas de cada una** —van a cadencia mensual, trimestral y semestral—: hay que coger las versiones ancladas que el Área de Descarga publica como entradas de dependencia. El cargador lee **todos** los ficheros que casan con cada patrón, no uno: leyendo uno solo, el nombre en español de un concepto internacional se pierde y el `CodeSystem` sale publicado con el número del código puesto de nombre, sin un aviso (`adr-0039`).
- **El mismo bloqueo deja abierta una comprobación que sí importa.** El SNS publica un *refset* de 617 tipos de muestra de laboratorio para la misma variable que aquí se enumera con diez códigos escritos a mano, y **entre las dos listas no hay ninguna relación declarada**. Que la local sea más corta es correcto —es la oferta del laboratorio, no el catálogo nacional—; que no se pueda comprobar, no. La comprobación necesita el fichero de miembros del *refset*, que está en la misma release (`adr-0040`).

13.2. Modelado que se dejó fuera a propósito

2. `Observation.device` sigue vacío. El identificador del analizador llega en `OBX-18` y no se proyecta: para apuntar ahí haría falta un inventario de analizadores como recursos `Device`, y crearlo desde el canal convertiría al motor en autoridad de un inventario que solo conoce de oídas. La identidad del aparato **no se pierde** —está en el mensaje original archivado—, pero el recurso no la lleva.

3. El `Patient` exportado no lleva municipio, aunque `D23` lo prevea. No es del exportador: **el agregado del dominio no modela la dirección**, así que la proyección la descarta al escribir. Comprobado en vivo: se manda un `Patient` con `address.city = Sevilla` y el GET posterior lo devuelve sin `address`. Consecuencia práctica: con la cohorte de hoy **no se puede dibujar un mapa de casos**, que es la mitad de para qué sirve una cohorte de vigilancia. Arreglarlo es trabajo de dominio.

4. Un facultativo duplicado en el directorio burla la doble validación. La segunda firma se compara por la referencia literal, así que la misma persona dada de alta dos veces podría poner las dos. Es un problema del directorio —dato maestro sin agregado detrás—, no de la regla.

13.3. Infraestructura montada para demostrar

5. El registro de esquemas lo prueban los tests en memoria; el camino HTTP solo se ha visto en vivo. Contra el compose funciona —los cuatro sujetos aparecen al primer envío y la compatibilidad sale en `BACKWARD`—, pero **ninguna puerta automática lo cubre**: un fallo de serialización o de configuración del registro no lo vería ningún test.

6, 7, 8. Instancia única en lo asíncrono, `NDJSON` en disco local y consola del motor sin autenticación: descritos en el capítulo 12.

13.4. Cliente y pantallas

9. La app del ciudadano no se ha ejecutado en un dispositivo. El flujo `SMART` se recorrió con las mismas peticiones que hace la app, pero **nadie ha visto la pantalla** en un emulador ni en un móvil: no hay constancia de que se pinte bien, de que el navegador embebido del `authorize` vuelva, ni de que la resolución de `10.0.2.2` —escrita en el código— funcione contra un emulador de verdad. Eso sigue fuera y **ninguna CI lo puede cubrir**: hace falta una persona mirando.

Lo que ya **no** está abierto es la otra mitad: la CI **empaqueta** la app en `release`, así que un fallo del manifiesto o de la configuración de red sí lo ve una puerta. La primera ejecución de esa puerta ya cazó uno, y la instalación de la dependencia cazó otro: **el paquete del SDK de Android no se llama como el destino de compilación** —Gradle pide `android-37` y Google publica `platforms;android-37.0`—, así que instalarlo termina en error a secas, sin decir que ese paquete no existe.

10. Ningún cliente llama a `$validar`, ni emite informes, ni gestiona la bandeja de EDO. Las tres operaciones funcionan, también con la seguridad puesta, pero **la web no tiene pantalla** para ninguna: el circuito completo solo lo recorre un guion. Es trabajo de pantalla, no de contrato; los ámbitos existen en el *realm*.

11. Una declaración rechazada no se reintenta sola y nadie la recoge. Es lo correcto —reenviar veinte veces algo rechazado por el contenido no lo arregla— pero deja **una obligación legal incumplida esperando a que una persona la mire**, y hoy solo sale por el log y por una búsqueda de `Task`. Lo mismo con las que agotan los intentos y con las vencidas.

13.5. Cobertura y garantías que se saben incompletas

12. La detección de EDO se apoya en que validar bloquea sin terminología. Hoy no existe el camino «se validó y no se preguntó si era declarable», porque una de las dos consultas al catálogo lanza si el servidor no está. Es una dependencia entre dos reglas que viven en clases distintas y **no hay nada que la haga cumplirse**: si mañana alguien decide que un resultado sin umbral conocido se puede validar igual, esta garantía desaparece en silencio. Está escrita en la documentación de las dos.

13. El corpus del reconciliador no sale del generador de datos sintéticos. El reconciliador sí se ha ejercitado sobre un laboratorio con volumen —300 pacientes, las cuatro formas de divergir provocadas y reparadas—, pero el corpus lo escribe un test recorriendo el circuito, no el generador.

14. Cargar el corpus del generador dentro del laboratorio no lo hace nadie. El generador **escribe ficheros y no publica en la API**. No estaba en ningún hito, y la base arranca vacía a propósito: la pantalla de alta busca al paciente por su número de historia y, si no consta, lo da de alta ahí mismo, que es lo que hace el mostrador de un laboratorio privado con quien llega por primera vez.

13.6. El siguiente trabajo natural, si alguien continúa

Por este orden, y ninguno es obligatorio:

1. **Las tres pantallas que faltan** (entrada 10). Validar, emitir informe y gestionar la bandeja de EDO funcionan por API; es el camino más corto de aquí a algo enseñable.
 2. **La app en un dispositivo de verdad** (entrada 9). Es lo único que ninguna CI puede cubrir.
 3. **La dirección en el agregado del dominio** (entrada 3). Hasta que el paciente la tenga, la cohorte exportada no permite dibujar un mapa de casos.
 4. **Cargar SNOMED** (entrada 1), el día que exista el fichero.
-

Capítulo 14

Procedencia de este documento

Este documento se escribió al cerrar el proyecto, para que el conocimiento sobreviviera al desmontaje de la maquinaria con la que se construyó: los ocho ficheros de instrucciones para el agente de desarrollo (`CLAUDE.md`), su contrato operativo (`AGENTS.md`), el plan de trabajo (`docs/PLAN.md`) y la configuración del entorno. El inventario de qué contenía cada uno y dónde ha quedado está en `docs/inventario-desmontaje.md`, junto a la fuente de este documento.

Lo que sigue vivo en el repositorio, y amplía lo de aquí:

Fichero	Qué añade
<code>README.md</code>	La puerta de entrada operativa: cómo levantarlo, orden por orden
<code>docs/disenio.md</code>	El porqué de las decisiones D1 a D20, con las fuentes normativas españolas citadas una a una
<code>docs/adr/</code> (45)	Cada decisión de arquitectura con su contexto, sus alternativas descartadas y sus consecuencias
<code>docs/destilacion.md</code>	Qué aporta cada ADR más allá de este proyecto, y a qué documento de convenciones iría
El historial de git	Commits firmados, atómicos y en español, con la identidad del autor y sin ningún trailer ajeno

Este PDF se regenera con `docs/generar-memoria-pdf.sh`, que rasteriza los diagramas Mermaid y compone el documento con pandoc y XeLaTeX dentro de contenedores, de modo que no hace falta instalar nada más que Docker.